CIPHERNEST: AI-NATIVE APPLICATION SECURITY PLATFORM

A PROJECT REPORT

Submitted By

VAIRAMUTHU M (110824405016)

In partial fulfilment for the award of the degree

Of

MASTER OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

JAYA ENGINEERING COLLEGE, THIRUNINRAVUR

ANNA UNIVERSITY : CHENNAI 600 025

JULY 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project work **"CIPHERNEST: AI-NATIVE APPLICATION SECURITY PLATFORM"** is the Bonafide work of **VAIRAMUTHU M (110824405016)** who carried out the project under my supervision.

Mr. J. LIN EBY CHANDRA,
Head of the Department,
Department of CSE,
Jaya Engineering College,
Thiruninravur – 602 024.

Mr. J. LIN EBY CHANDRA,
Supervisor,
Department of CSE,
Jaya Engineering College,
Thiruninravur – 602 024.

ANNA UNIVERSITY : CHENNAI 600 025

VIVA VOCE EXAMINATION

The viva-voce examination of the project work titled **"CIPHERNEST: AI-NATIVE APPLICATION SECURITY PLATFORM"** submitted by **VAIRAMUTHU M (110824405016)** held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

CipherNest is an AI-native application security platform that addresses risks introduced by LLMs, AI agents, MCP servers, modern software supply chains, and cloud-native architectures. It unifies analysis across source code, AI prompts, agent workflows, MCP configurations, dependencies, CI/CD pipelines, containers, and infrastructure-as-code to detect vulnerabilities and misconfigurations specific to AI-driven systems. CipherNest combines static code analysis, AST inspection, taint-flow analysis, heuristic detectors, attack-path correlation, and compliance checks to find OWASP Top 10 and OWASP LLM Top 10 issues, exposed secrets, dependency-based threats, excessive agent permissions, insecure MCP setups, and runtime tool misuse. Implemented as a modular Next.js/TypeScript frontend with Node.js services and PostgreSQL storage, the platform produces quantitative security scores and grades, and generates attack graphs that correlate findings to visualize potential exploitation and privilege escalation paths. Additional capabilities include runtime monitoring, threat intelligence feeds, dependency reachability analysis, and automated remediation recommendations. Experimental compliance assessments map controls to frameworks such as OWASP ASVS and organizational policies.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS, ABBREVIATIONS & DEFINITIONS

SYMBOLS

▭ — External Entity

○ — Use Case

→ — Data Flow

◇ — Decision Point

● — Database

⇄ — Bidirectional Communication

ABBREVIATIONS

AI — Artificial Intelligence

LLM — Large Language Model

MCP — Model Context Protocol

AST — Abstract Syntax Tree

API — Application Programming Interface

SAST — Static Application Security Testing

DAST — Dynamic Application Security Testing

OWASP — Open Worldwide Application Security Project

ASVS — Application Security Verification Standard

CI/CD — Continuous Integration / Continuous Deployment

RBAC — Role Based Access Control

PKCE — Proof Key for Code Exchange

CVE — Common Vulnerabilities and Exposures

DEFINITIONS

AI Application Security:
The practice of identifying and mitigating security risks associated with Large Language Models, AI agents, prompts, and AI-enabled software systems.

Model Context Protocol (MCP):
An open protocol that enables AI agents to securely interact with external tools, services, databases, and applications.

Static Application Security Testing (SAST):
A methodology that analyzes source code without executing the application to identify potential vulnerabilities.

Attack Graph:
A visual representation of possible attack paths that an adversary may exploit to compromise a system.

Supply Chain Security:
The process of securing software dependencies, packages, libraries, build pipelines, and third-party components from compromise.

Runtime Shield:
A security monitoring layer that inspects application behavior and tool interactions during execution to detect malicious activities.

Compliance Assessment:
The process of evaluating security controls against frameworks such as OWASP ASVS and organizational security requirements.

Agent Security:
The protection of autonomous AI agents against excessive permissions, prompt injection, tool abuse, and unauthorized actions.

Threat Intelligence:
Security information collected from external sources regarding vulnerabilities, exploits, indicators of compromise, and emerging threats.

OWASP LLM Top 10:
A security framework defining the most critical risks affecting Large Language Model applications and agentic systems.

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

1.1 SYSTEM OVERVIEW

CipherNest is an AI-native Application Security Platform developed to address the emerging security challenges associated with Large Language Models (LLMs), AI agents, Model Context Protocol (MCP) ecosystems, software supply chains, and modern cloud-native applications. As organizations increasingly adopt AI-powered systems and autonomous agents, traditional application security tools are often unable to detect vulnerabilities unique to agentic architectures, prompt-driven workflows, excessive tool permissions, insecure MCP configurations, and AI-assisted attack vectors.

CipherNest provides a unified platform for identifying, analyzing, and mitigating security risks across multiple layers of modern software systems. The platform combines source code security analysis, OWASP Top 10 vulnerability detection, OWASP LLM Top 10 assessment, secrets discovery, dependency risk analysis, AI agent security auditing, MCP security validation, runtime protection, and compliance assessment into a single integrated framework.

The system is implemented using a modular architecture consisting of a Next.js web interface, TypeScript services, Node.js backend components, and PostgreSQL-based storage. A centralized security engine coordinates specialized scanning modules that inspect source code repositories, AI prompts, agent workflows, MCP configurations, CI/CD pipelines, containerized environments, and infrastructure-as-code resources. The platform generates security findings, attack graphs, risk scores, and remediation recommendations that assist developers and security teams in strengthening application security posture.

CipherNest further incorporates runtime protection capabilities through a security-aware gateway that monitors tool interactions between AI agents and MCP servers. By combining static analysis, attack path correlation, threat intelligence, and runtime monitoring, the platform delivers trustworthy security insights for modern AI-enabled applications.

1.2 OBJECTIVE

The primary objective of CipherNest is to develop a comprehensive AI Application Security Platform capable of identifying vulnerabilities and security misconfigurations across software applications, AI systems, and agentic environments while maintaining high detection accuracy and actionable remediation guidance.

The specific objectives are:

- To analyze source code repositories and detect vulnerabilities aligned with OWASP Top 10 security risks.
- To identify security weaknesses in Large Language Model integrations based on OWASP LLM Top 10 guidelines, including prompt injection, insecure output handling, excessive agency, and model abuse scenarios.
- To assess AI agents and Model Context Protocol (MCP) configurations for excessive permissions, insecure tool access, unauthorized actions, and potential attack vectors.
- To perform software supply chain analysis by evaluating third-party dependencies, lockfiles, installation scripts, and known vulnerabilities from public security databases.
- To generate attack graphs that correlate vulnerabilities and visualize potential exploitation paths, privilege escalation routes, and security impact.
- To provide runtime monitoring capabilities through a gateway that validates tool interactions and detects suspicious behaviors during execution.
- To evaluate compliance requirements using security control mappings and generate security posture scores for developers and organizations.
- To create an extensible architecture that can support future enhancements such as AI-driven remediation, autonomous security agents, advanced threat intelligence integration, and enterprise-scale deployment.

1.3 SYSTEM STUDY

1.3.1 Existing System

Current application security solutions focus primarily on traditional software vulnerabilities and often lack dedicated support for modern AI-enabled systems. Popular Static Application Security Testing (SAST) platforms such as SonarQube, Semgrep, and Snyk

provide source code analysis capabilities but offer limited visibility into AI agent workflows, prompt security risks, and MCP ecosystems.

Most existing tools evaluate application code, dependencies, or infrastructure independently. Security analysis for AI prompts, agent permissions, tool access control, and runtime MCP interactions is either absent or provided through isolated solutions. As a result, organizations must use multiple products to secure different layers of their applications.

Furthermore, traditional security scanners are not designed to address emerging threats such as prompt injection attacks, excessive AI agent privileges, insecure MCP configurations, tool output manipulation, model abuse, and AI-assisted supply chain attacks. The lack of a unified security platform creates operational complexity, fragmented visibility, and inconsistent risk management.

1.3.2 Literature Survey

Recent research highlights the growing importance of securing AI-enabled software systems. Studies on Static Application Security Testing demonstrate that automated code analysis can identify a significant percentage of common software vulnerabilities before deployment. Research on taint analysis and abstract syntax tree (AST) inspection has shown improved effectiveness in detecting injection attacks, insecure data flows, and privilege escalation paths.

The emergence of Large Language Models has introduced new categories of security risks. Research published on OWASP LLM Top 10 vulnerabilities identifies prompt injection, insecure output handling, excessive agency, sensitive information disclosure, and model theft as critical concerns. Studies further indicate that AI agents with unrestricted tool access may perform unintended actions that compromise confidentiality, integrity, and availability.

Software supply chain security has also become a major research area due to the increasing use of open-source components. Vulnerabilities in dependencies, malicious packages, dependency confusion attacks, and compromised build pipelines continue to affect software ecosystems worldwide. Research suggests that dependency analysis combined with threat intelligence improves early detection of supply chain risks.

Advantages of Existing Approaches:

- Effective detection of common software vulnerabilities.
- Mature support for code scanning and dependency analysis.
- Availability of standardized security frameworks such as OWASP and ASVS.
- Extensive open-source security databases and threat intelligence feeds.

Limitations of Existing Approaches:

- Limited support for AI-specific security risks.
- Lack of comprehensive MCP security validation.
- Insufficient visibility into agent permissions and runtime tool interactions.
- Fragmented security workflows requiring multiple tools.
- Minimal support for attack-path correlation across different security domains.

1.3.3 Proposed System

CipherNest introduces a unified AI Application Security Platform that integrates security analysis across source code, AI systems, software supply chains, runtime environments, and compliance frameworks. Users can connect repositories, scan application code, analyze AI agent configurations, validate MCP settings, and assess dependency risks through a single platform.

The Security Engine orchestrates multiple scanning modules including code security analysis, secrets detection, dependency assessment, AI security validation, agent auditing, MCP security checks, CI/CD security inspection, container security analysis, and infrastructure security evaluation. Findings from all modules are aggregated and processed through a centralized risk-scoring framework.

The Attack Graph Engine correlates vulnerabilities using dependency relationships, privilege models, and reachability analysis to generate visual representations of possible attack paths. The Runtime Shield monitors tool interactions and applies security policies to prevent malicious operations, data exfiltration attempts, command injection attacks, and unauthorized access.

The Compliance Engine evaluates findings against established security standards and generates compliance reports that assist organizations in measuring security posture. Experimental evaluation demonstrates improved visibility across AI-enabled applications while providing actionable recommendations for vulnerability remediation and risk reduction.

1.4 ORGANIZATION OF THE REPORT

This report presents the complete design, implementation, and evaluation of CipherNest.

Chapter 2 describes the Software Requirements Specification, including functional requirements, system features, external interfaces, and non-functional requirements.

Chapter 3 explains the Software Design, covering system architecture, module decomposition, component interactions, database design, and user interface design.

Chapter 4 discusses the implementation details of the web platform, security engine, attack graph engine, MCP security modules, runtime protection mechanisms, and compliance assessment framework.

Chapter 5 presents the testing methodology, test cases, validation procedures, and performance evaluation results.

Chapter 6 discusses the results and observations obtained during experimentation, benchmark analysis, and security assessment.

Chapter 7 concludes the work and outlines future enhancements including advanced security reasoning, autonomous remediation, enterprise-scale deployment, and AI-driven threat intelligence integration.

CHAPTER 2

SOFTWARE REQUIREMENTS SPECIFICATION

2. SOFTWARE REQUIREMENTS SPECIFICATION

2.1 EXTERNAL INTERFACE REQUIREMENTS

User Interfaces:

CipherNest provides a modern web-based security dashboard developed using Next.js and TypeScript. The platform enables developers and security teams to scan source code repositories, analyze AI applications, evaluate MCP configurations, and monitor security posture through an intuitive interface.

The primary dashboard displays security scores, vulnerability summaries, attack graphs, compliance status, and remediation recommendations. Users can initiate repository scans, review findings by severity level, visualize attack paths, and monitor security trends through interactive charts and reports.

The Attack Graph interface presents correlated vulnerabilities and exploitation paths in graphical form. The Compliance Dashboard displays OWASP and ASVS coverage metrics. The MCP Security Dashboard provides visibility into agent permissions, tool access, and runtime security events.

Hardware Interfaces:

CipherNest operates as a web-based platform and requires only a standard computing device with a modern web browser. The system supports desktop and laptop environments running Chrome, Firefox, Edge, or Safari. Minimum requirements include 4GB RAM, dual-core processor, and stable internet connectivity for repository analysis and threat intelligence lookups.

Software Interfaces:

- Next.js Web Application
- Node.js Backend Services
- PostgreSQL Database
- GitHub Repository APIs
- OpenSSF Scorecard APIs
- OSV Vulnerability Database

- GitHub Security Advisories
- MCP Servers and Agent Frameworks
- OWASP Compliance Frameworks

Communication Interfaces:

All communications utilize HTTPS with TLS 1.3 encryption. REST APIs are used for frontend-backend communication. WebSocket channels provide real-time scan progress updates and security notifications. Database communications use SSL-encrypted PostgreSQL connections.

2.2 SYSTEM FEATURES

2.2.1 Security Scanning Engine

Description:

The Security Scanning Engine serves as the core component of CipherNest. It analyzes source code repositories and identifies security vulnerabilities based on OWASP Top 10 guidelines, custom security rules, and static analysis techniques.

User Flow:

User selects repository → Scan initiated → Files analyzed → Vulnerabilities detected → Findings categorized by severity → Security score generated → Results displayed on dashboard.

Functional Requirements:

REQ-1: Support scanning of JavaScript, TypeScript, Python, and configuration files.

REQ-2: Detect OWASP Top 10 vulnerabilities.

REQ-3: Generate findings with severity classification.

REQ-4: Produce security scores and grades.

REQ-5: Export results in JSON and SARIF formats.

2.2.2 LLM Security Analysis Module

Description:

The LLM Security Module evaluates AI-enabled applications against OWASP LLM Top 10 security risks. The module identifies prompt injection vulnerabilities, insecure output handling, excessive agency, sensitive information disclosure, and model abuse scenarios.

Functional Requirements:

REQ-1: Detect prompt injection patterns.

REQ-2: Identify insecure LLM output handling.

REQ-3: Detect sensitive data exposure in prompts.

REQ-4: Validate model configuration security.

REQ-5: Generate OWASP LLM Top 10 compliance reports.

2.2.3 AI Agent Security Module

Description:

This module evaluates autonomous AI agents and agent frameworks for excessive permissions, unrestricted tool access, missing human approvals, and dangerous execution paths.

Functional Requirements:

REQ-1: Identify unrestricted tool permissions.

REQ-2: Detect unsafe file system access.

REQ-3: Detect unsafe database write operations.

REQ-4: Validate human approval workflows.

REQ-5: Generate agent security posture reports.

2.2.4 MCP Security Assessment Module

Description:

The MCP Security Assessment Module analyzes Model Context Protocol configurations and runtime interactions. It detects dangerous permissions, insecure tool registrations, wildcard access policies, and shadow MCP servers.

Functional Requirements:

REQ-1: Validate MCP configurations.

REQ-2: Detect excessive tool permissions.

REQ-3: Identify unauthorized MCP servers.

REQ-4: Analyze runtime tool calls.

REQ-5: Generate MCP security findings.

2.2.5 Dependency & Supply Chain Security Module

Description:

This module evaluates third-party packages, dependencies, lockfiles, and installation scripts for supply-chain risks and known vulnerabilities.

Functional Requirements:

REQ-1: Identify vulnerable dependencies.

REQ-2: Perform lockfile reachability analysis.

REQ-3: Detect dependency confusion risks.

REQ-4: Scan install-time scripts.

REQ-5: Integrate vulnerability intelligence feeds.

2.2.6 Runtime Shield Module

Description:

The Runtime Shield continuously monitors runtime behavior and tool interactions between AI agents and MCP servers. The module enforces security policies and blocks suspicious actions.

Functional Requirements:

REQ-1: Monitor runtime tool executions.

REQ-2: Detect command injection attempts.

REQ-3: Detect data exfiltration attempts.

REQ-4: Generate runtime audit logs.

REQ-5: Apply policy-based blocking rules.

2.2.7 Compliance Assessment Engine

Description:

The Compliance Engine evaluates detected findings against security frameworks and organizational requirements.

Functional Requirements:

REQ-1: Assess OWASP compliance.

REQ-2: Calculate ASVS control coverage.

REQ-3: Generate compliance scorecards.

REQ-4: Identify failed controls.

REQ-5: Produce audit-ready reports.

2.3 OTHER NON-FUNCTIONAL REQUIREMENTS

Performance Requirements:

- Repository scan completion within 60 seconds for medium-sized projects.
- Dashboard load time below 2 seconds.
- Security score generation below 5 seconds.
- Attack graph generation below 10 seconds.
- Support for 500 concurrent scan sessions.

Security Requirements:

- JWT-based authentication.
- HTTPS/TLS 1.3 enforcement.
- Role-Based Access Control (RBAC).
- Input validation and sanitization.
- Secure secret storage.

Availability Requirements:

- 99.5% service availability.
- Automated backup procedures.
- Fault-tolerant deployment architecture.
- Graceful error handling.

Usability Requirements:

- Responsive web interface.
- Clear severity-based findings presentation.
- Interactive attack graph visualization.
- Security reports understandable by developers and analysts.

Scalability Requirements:

- Horizontal scaling support.
- Multi-repository scanning capability.
- Modular architecture for future expansion.

CHAPTER 3
SOFTWARE DESIGN

3. SOFTWARE DESIGN

3.1 ARCHITECTURAL DESIGN

3.1.1 System Architecture

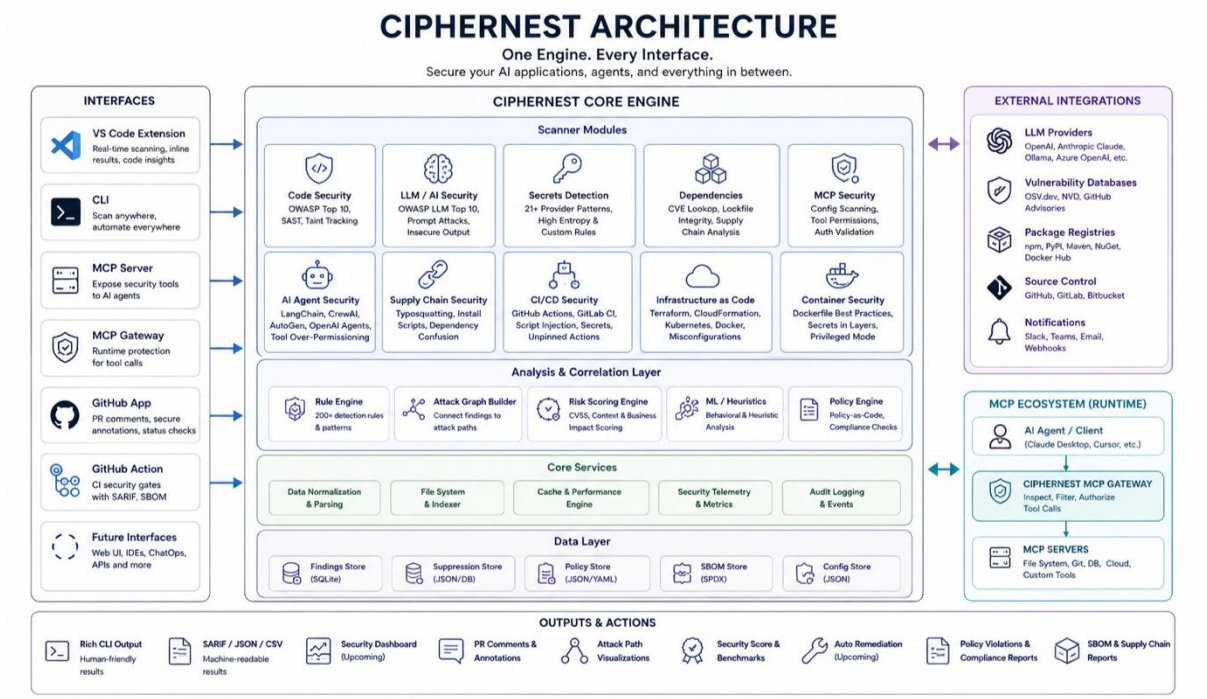

Fig 3.1 CipherNest System Architecture

The Presentation Layer forms the entry point of the CipherNest platform, implemented as a responsive web dashboard utilizing Next.js, React, and TypeScript. It is designed to provide security analysts, developers, and administrators with an intuitive, unified interface to interact with the platform's security services. The dashboard dynamically renders security findings, visualizes attack graphs, aggregates real-time scan metrics, displays compliance scorecards, and manages user accounts. Using modern React component design and client-side routing, the presentation layer provides interactive severity-based filters, dynamic scan triggers, and visual path inspections without unnecessary page reloads. To facilitate real-time security events monitoring, the dashboard establishes WebSocket connections for receiving instant alerts from the runtime shield and progress updates from ongoing repository scans. Security is enforced client-side by validating inputs before API transmission and checking user permissions before rendering restricted administrative panels.

The API Gateway Layer serves as the central middleware of the platform, built using Node.js and the Express framework. It exposes RESTful API endpoints and WebSocket channels that secure and coordinate client-server interactions. The API Gateway is responsible for handling user authentication (via JWT-based sessions), enforcing Role-Based Access Control (RBAC), validating request payloads against strict Zod schemas, and rate-limiting public endpoints to protect against Denial of Service (DoS) attacks. Furthermore, the gateway orchestrates requests between the frontend dashboard and the downstream security engines, ensuring that data is correctly routed and sanitized before reaching the persistent storage. By decoupling the presentation layer from the core security logic, the API Gateway maintains a clean separation of concerns and acts as a security boundary that filters out malicious payloads and unauthorized requests.

The Security Engine Layer forms the core analytical powerhouse of CipherNest. It comprises five specialized sub-engines: the Security Scanning Engine, the LLM Security Engine, the Agent & MCP Security Engine, the Attack Graph Engine, and the Runtime Shield Engine. These engines run concurrently as background services, communicating with the gateway and database. The Security Scanning Engine runs static code scanners and dependency analyzers on repositories. The LLM Security Engine parses and evaluates prompts against injection databases. The Agent & MCP Security Engine validates agent tool registrations and MCP configurations. The Attack Graph Engine correlates all results into exploitability paths, and the Runtime Shield Engine enforces real-time blocking policies on active agent-to-tool operations. Together, they form a cohesive analysis fabric.

The Data Layer is the foundational storage tier of the platform, powered by a PostgreSQL relational database. It serves as the persistent system of record, storing structured data including user credentials, repository metadata, vulnerability scan reports, security findings, compliance scorecards, runtime audit logs, and generated attack graph paths. The database schema is designed with foreign key constraints, unique indexes, and check constraints to guarantee referential integrity and data consistency. Row-Level Security (RLS) policies are enforced at the database level to ensure that users can only access data belonging to their respective organizations or repositories, preventing unauthorized cross-tenant data access. Database connections are secured using SSL, and sensitive credentials such as API keys, secrets, and private repository tokens are encrypted at rest using industry-standard AES-256 encryption.

3.1.2 Use Case Diagram

The Use Case Diagram defines the functional scope of the CipherNest platform, capturing the interactions between the system boundaries and the main actors: Developer, Security Analyst, and System Administrator. The Developer primarily interacts with repository scanning, viewing vulnerability findings, and checking compliance reports. These interactions allow developers to remediate code-level flaws early in the software development lifecycle. The Security Analyst possesses broader authorization, focusing on advanced auditing tasks such as analyzing AI prompt templates, evaluating Model Context Protocol (MCP) server configurations, generating threat graphs, and auditing runtime events. The System Administrator manages user provisioning, configures RBAC policies, and reviews audit logs.

Use-case relationships are represented using include and extend associations. For instance, the "Scan Repository" use case includes "View Findings" as its default outcome, whereas "Generate Attack Graph" extends "View Findings" conditionally when multiple correlated vulnerabilities are identified. "Analyze MCP Config" includes "Analyze AI Agent" to ensure tool schemas align with agent permissions. This structured representation ensures clear boundary definitions and maps functional requirements directly to user roles.

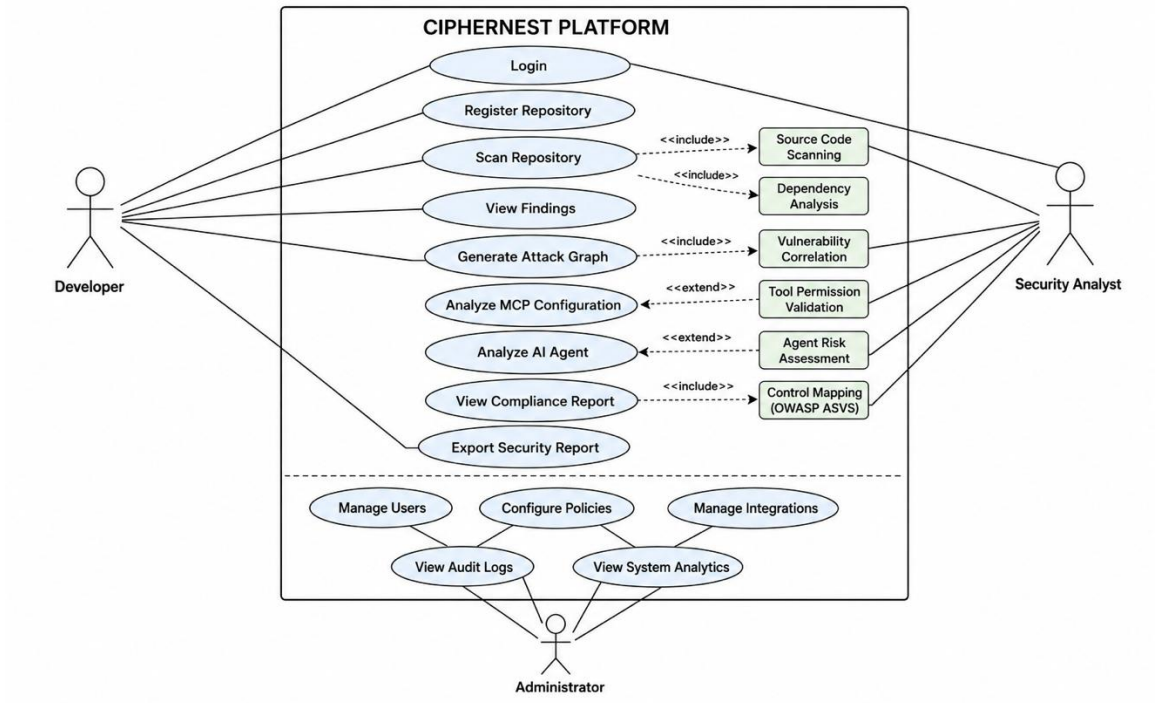

Fig 3.2 Use Case Diagram

The system defines three primary actors who interact with the security platform:

Developer: Interacts with the platform to initiate repository scans, view vulnerability reports, check compliance statuses, and follow remediation guidelines.

Security Analyst: Interacts with the platform to evaluate AI prompt security, audit Model Context Protocol (MCP) server settings, analyze generated attack graphs, review compliance metrics, and export audit reports.

System Administrator: Manages user accounts, configures platform security policies, monitors system performance, and reviews global audit logs.

The platform implements the following core use cases to support security analysis:

Login: Authenticates system actors and establishes secure, role-restricted user sessions.
Scan Repository: Clones source code repositories and coordinates static vulnerability analysis, secrets detection, and dependency risk mapping.
View Findings: Displays and filters scan results according to severity levels and vulnerability categories.
Generate Attack Graph: Correlates vulnerabilities across repositories, agents, and dependencies to map potential exploitation chains.
Analyze MCP Config: Validates Model Context Protocol JSON files for excessive permissions, wildcard access, or transport vulnerabilities.
Analyze AI Agent: Audits prompt templates for injection risks, excessive agency, and data exposure vulnerabilities.
View Compliance Report: Renders security posture scores mapped against OWASP Top 10 and ASVS guidelines.
Export Report: Produces structured PDF and JSON reports suitable for compliance and security audits.
Manage Users: Allows administrators to create, update, or revoke access credentials and roles.

3.1.3 Sequence Diagram

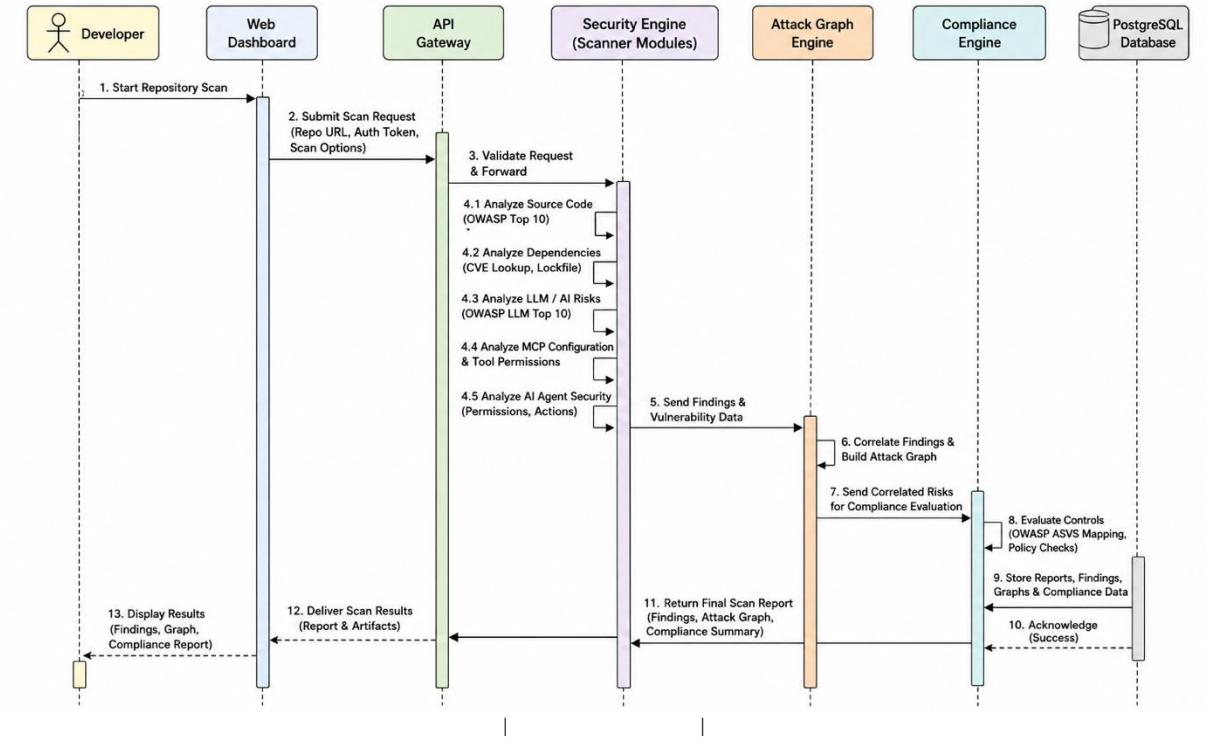

Fig 3.3 Sequence Diagram

The Sequence Diagram describes the end-to-end execution flow of a repository security scan, tracking the temporal interactions across the platform's components from the user interface down to the persistent database. The flow begins when a Developer or Security Analyst triggers a "Scan Repository" command from the Next.js Web Dashboard. The dashboard translates this interaction into an HTTPS POST request, which is dispatched to the Express API Gateway. The API Gateway intercepts the request, validates the user's JWT session token, and inspects the payload for compliance with the schema.

Once authorized, the Gateway forwards the scan instruction to the central Security Scanning Engine. The Security Scanning Engine pulls the target repository files, coordinates static code analysis, and flags vulnerabilities and hardcoded secrets. Upon completing the raw code scan, the Security Scanning Engine forwards the findings to the Attack Graph Engine. The Attack Graph Engine correlates these code-level findings with the application's AI agent tool definitions and dependency chains to identify reachability paths, outputting a complete attack graph.

Next, the aggregated results are passed to the Compliance Assessment Engine, which maps each finding to specific OWASP Top 10, OWASP LLM Top 10, and ASVS security controls to calculate a compliance score. The Compliance Engine then formats the final report and sends it to the PostgreSQL Database for persistence. The database returns a write confirmation to the API Gateway. Finally, the Gateway delivers a success message to the Web Dashboard, which dynamically renders the updated security score, findings list, and visual attack graph path for the user.

3.2 DECOMPOSITION DESCRIPTION

The CipherNest platform is decomposed into several modular backend engines and services, each encapsulating specific functional concerns. This decomposition isolates analytical processes, enforces runtime control boundaries, and supports horizontal scaling. The division of roles between code scanners, LLM sanitizers, permission validators, path correlators, runtime guards, and compliance assessment components is described in detail below.

3.2.1 Security Scanning Engine

The Security Scanning Engine is responsible for static application security testing (SAST), secrets detection, and dependency risk auditing. When a repository is linked, this engine clones the codebase to a secure scratch directory, reads project configurations, and runs rules-based scanners. It parses abstract syntax trees (ASTs) of JavaScript, TypeScript, and Python source files to identify common code flaws, while checking configurations for plain-text secrets and scanning dependencies against vulnerability databases.

Responsibilities of the Security Scanning Engine include:

Source code analysis: Performs static analysis of application source files to detect code-level flaws, injection risks, and syntax vulnerabilities.

Secrets detection: Scans repository history and configuration files for hardcoded API keys, private tokens, passwords, and encryption keys.

Dependency analysis: Audits project package manifests and lockfiles to map third-party components against known vulnerability databases.

Risk scoring: Evaluates findings dynamically to calculate a baseline severity score for each scanned codebase.

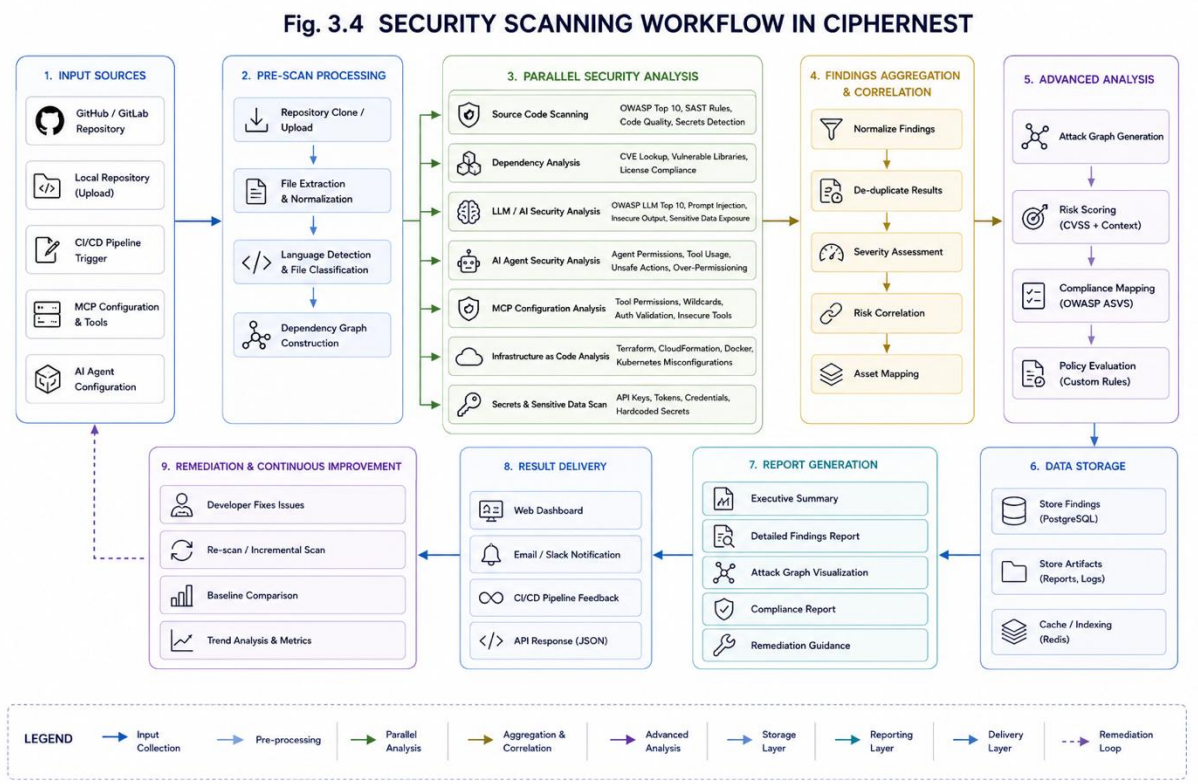

Fig 3.4 Security Scanning Workflow

3.2.2 LLM Security Engine

The LLM Security Engine is designed to identify vulnerabilities unique to Large Language Model integrations, aligning with the OWASP LLM Top 10. It parses prompt templates, agent system instructions, and input payload flows to evaluate weaknesses such as susceptibility to prompt injection, potential data exposure of sensitive inputs, model misuse, and excessive model agency. The engine evaluates prompt construction patterns and checks if LLM-derived parameters are sanitized before execution in external systems.

Responsibilities of the LLM Security Engine include:

Prompt Injection Detection: Scans user prompt templates and input data streams to block system prompt extraction and behavioral bypasses.

Sensitive Data Exposure: Identifies and redacts Personally Identifiable Information (PII), credentials, and intellectual property in prompt payloads.

Excessive Agency Detection: Audits LLM configurations to ensure models cannot execute system actions beyond their intended scope.

Model Misuse Analysis: Evaluates application configurations to prevent unauthorized fine-tuning, system prompt extraction, or model theft.

3.2.3 Agent & MCP Security Engine

The Agent & MCP Security Engine is a specialized auditor that inspects Model Context Protocol (MCP) configuration manifests and AI agent tool registries. It flags dangerous configuration patterns, such as wildcard tool permissions, shadow server definitions, or insecure communication paths, providing security teams with visibility into what actions autonomous agents are permitted to take on local and cloud resources.

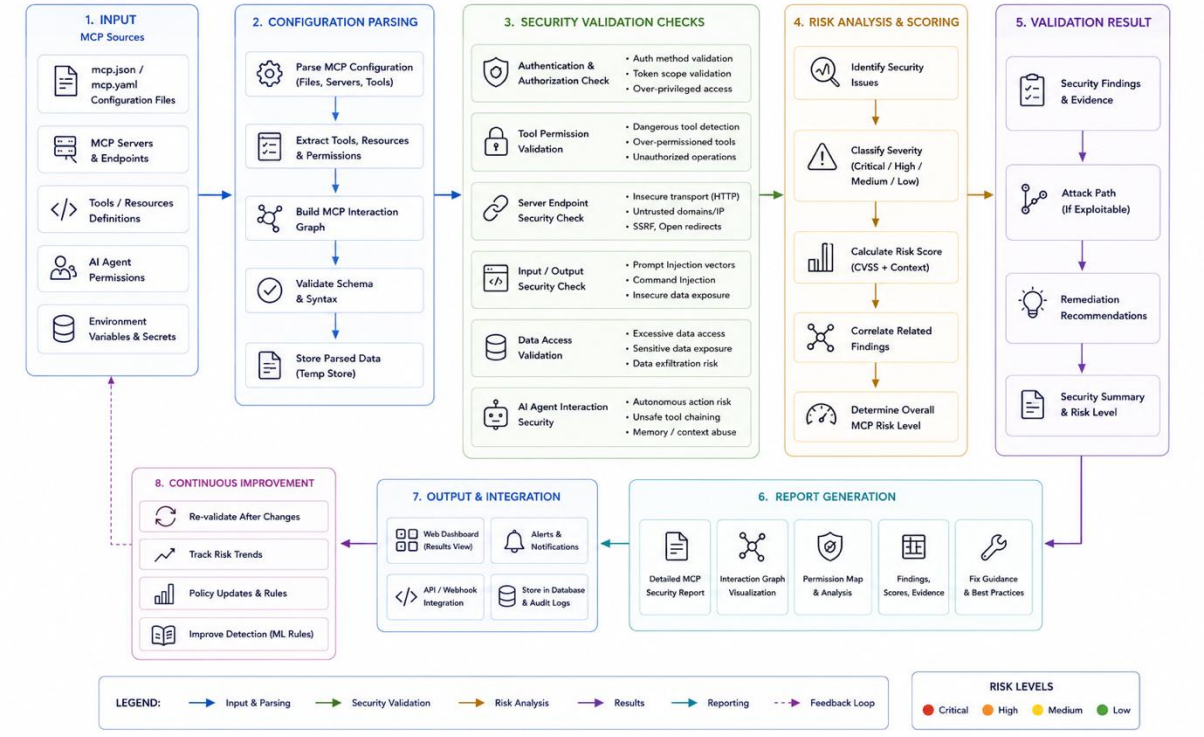

Fig 3.6 MCP Security Validation Flow

Responsibilities of the Agent & MCP Security Engine include:

Tool Permission Analysis: Audits tool schemas registered with AI agents to restrict execution privileges to minimal required operations.

MCP Validation: Assesses Model Context Protocol server configuration files for wildcard permissions, missing authentications, or weak transports.

Runtime Tool Monitoring: Tracks tool invocations in real-time to detect anomalous calling patterns or unexpected parameter payloads.

Excessive Permission Detection: Automatically alerts security teams when agent tools request wide directory, database, or execution access.

3.2.4 Attack Graph Engine

The Attack Graph Engine correlates findings across multiple security domains to construct exploitability paths. Instead of treating code bugs, agent permissions, and dependency risks in isolation, it models their relationships. For example, it tracks how a dependency flaw combined with a wildcard agent tool permission could allow an attacker to traverse from an LLM prompt down to an internal PostgreSQL database, calculating risk weights based on actual reachability.

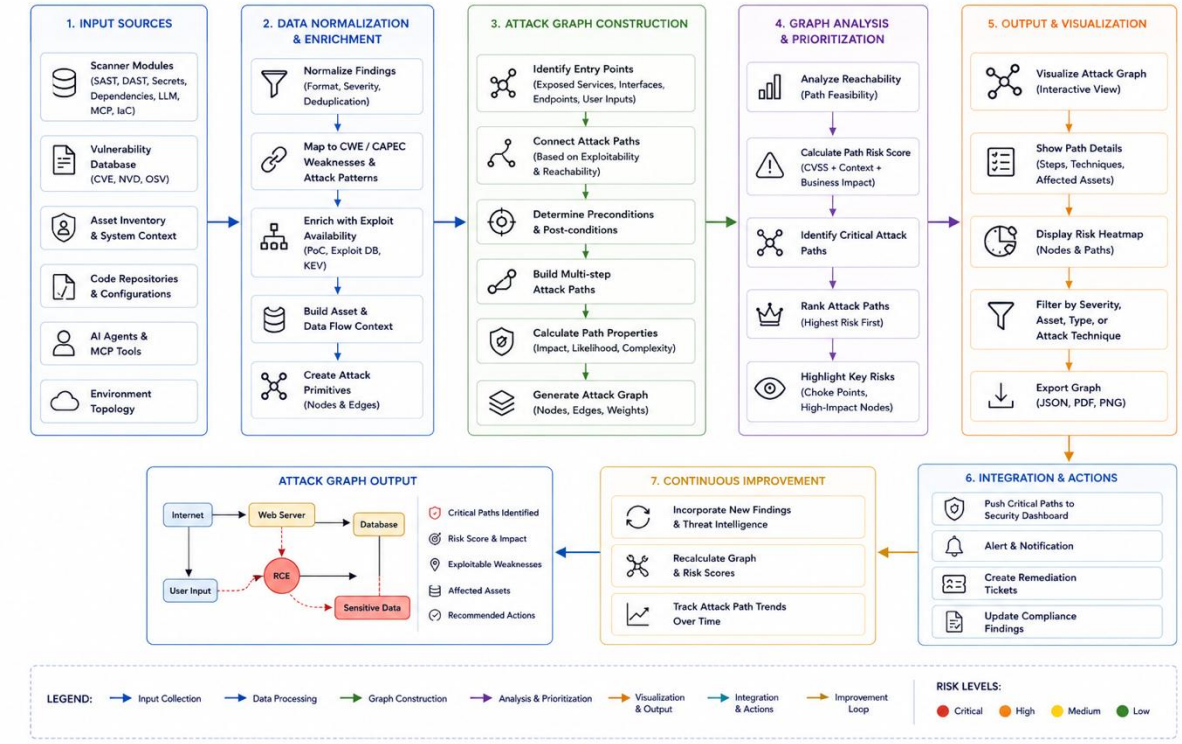

Fig 3.5 Attack Graph Generation Flow

Responsibilities of the Attack Graph Engine include:

Vulnerability Correlation: Chains code flaws, dependency risks, and agent permissions into unified, multi-step attack vectors.

Reachability Analysis: Simulates potential paths from external interfaces down to critical database systems and resources.

Privilege Escalation Detection: Evaluates whether an attacker could leverage compromised credentials or tool parameters to elevate privileges.

Risk Prioritization: Computes path-level risk weights to rank findings based on exploitability and threat impact.

3.2.5 Runtime Shield Engine

The Runtime Shield Engine acts as an inline policy enforcement point that intercepts agent-to-tool call payloads at runtime. It monitors execution requests, checking whether tool arguments contain command injections or unauthorized file paths, and intercepts server responses to prevent data exfiltration. If a policy violation is detected, the engine blocks the tool invocation and generates high-priority security audit trails.

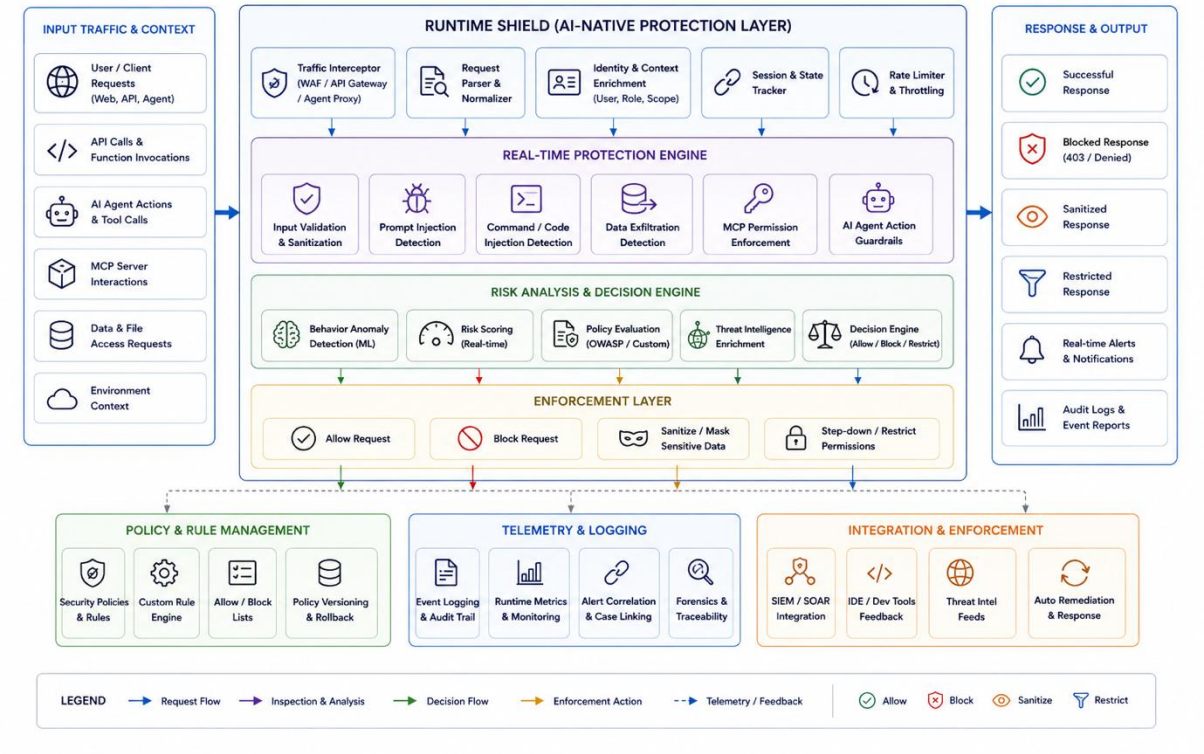

Fig 3.7 Runtime Shield Architecture

Responsibilities of the Runtime Shield Engine include:

Tool Call Monitoring: Interacts with the API layer to monitor and inspect all runtime messages sent between AI agents and Model Context Protocol servers.

Data Exfiltration Detection: Applies heuristic and content filters to identify sensitive data leaking through outgoing tool calls.

Command Injection Detection: Validates tool argument values against strict schemas to block OS command injection or SQL injection attacks.

Audit Logging: Generates high-fidelity audit trails documenting every tool invocation, input parameters, and security actions.

3.2.6 Compliance Assessment Engine

The Compliance Assessment Engine evaluates detected findings against public security frameworks and standard checklists. It maps vulnerabilities to specific OWASP Top 10 categories, OWASP LLM Top 10 rules, and OWASP Application Security Verification Standard (ASVS) control requirements. Based on this mapping, it derives numerical compliance scores and outputs audit scorecards for security teams.

Responsibilities of the Compliance Assessment Engine include:

OWASP Mapping: Links detected code, prompt, and configuration vulnerabilities to the OWASP Top 10 and OWASP LLM Top 10 categories.

ASVS Assessment: Maps system components against the OWASP Application Security Verification Standard (ASVS) control checklists.

Security Score Generation: Computes dynamic security grades based on control fulfillment, vulnerability density, and risk impact.

Compliance Reporting: Formats structural scorecards and audit reports reflecting compliance levels for regulatory audits.

3.3 COMPONENT DESIGN

The core software systems inside CipherNest are implemented using specialized, object-oriented component services. These components execute the core security logic, validate credentials, parse repository codes, build attack paths, evaluate MCP setups, and compile compliance coverage. The behavioral logic of each key component is defined through structured pseudocode blocks below.

3.3.1 Authentication Component

The Authentication Component verifies user-supplied credentials, generates JSON Web Tokens (JWT) for secure session tracking, and logs access events in the audit databases.

```
FUNCTION loginUser(email, password)
    // Validate email format and check non-empty password
    IF NOT validateEmailFormat(email) OR password IS EMPTY THEN
        RETURN Error("Invalid input parameters")
    END IF
```

```
    // Retrieve user record from database
    userRecord = database.query("SELECT * FROM users WHERE email = ?", email)
    IF userRecord IS NULL THEN
        RETURN Error("Authentication failed: User not found")
    END IF

    // Verify password hash
    passwordMatches = verifyHash(password, userRecord.password_hash)
    IF NOT passwordMatches THEN
        RETURN Error("Authentication failed: Incorrect password")
    END IF

    // Generate JSON Web Token (JWT) with user claims
    jwtClaims = {
        "userId": userRecord.id,
        "email": userRecord.email,
        "role": userRecord.role,
        "exp": currentTime() + 3600 // 1 hour expiration
    }
    jwtToken = generateJWT(jwtClaims, env.JWT_SECRET)

    // Create session audit log entry
    database.execute("INSERT INTO audit_logs (user_id, action, timestamp)
VALUES (?, 'LOGIN', ?)", userRecord.id, currentTime())

    RETURN Success(jwtToken)
END FUNCTION
```

3.3.2 Repository Scanning Component

The Repository Scanning Component coordinates static analysis, secrets scans, and dependency audits for connected repositories, calculating and saving findings.

```
FUNCTION scanRepository(repositoryId, branchName)
    // Retrieve repository details and access token
    repoRecord = database.query("SELECT * FROM repositories WHERE id = ?",
repositoryId)
    IF repoRecord IS NULL THEN
        RETURN Error("Repository not found")
    END IF

    // Clone repository to secure local scratch path
    scratchPath = cloneRepository(repoRecord.git_url, repoRecord.access_token,
branchName)
    IF scratchPath IS NULL THEN
        RETURN Error("Failed to clone repository")
    END IF

    // Initialize scan findings accumulator
    scanFindings = []

    // 1. Execute Static Application Security Testing (SAST)
    sastFindings = executeSastScanner(scratchPath)
    scanFindings.appendAll(sastFindings)

    // 2. Execute Secrets Detection Scanner
    secretsFindings = executeSecretsScanner(scratchPath)
    scanFindings.appendAll(secretsFindings)

    // 3. Execute Dependency Auditing Scanner
    dependencyFindings = executeDependencyScanner(scratchPath)
```

```
    scanFindings.appendAll(dependencyFindings)

    // Calculate aggregated security score (0-100 scale)
    securityScore = calculateSecurityScore(scanFindings)

    // Persist scan report in database
    reportRecord = database.execute(
        "INSERT INTO scan_reports (repository_id, branch, score, status,
created_at) VALUES (?, ?, ?, 'COMPLETED', ?)",
        repositoryId, branchName, securityScore, currentTime()
    )

    // Save individual findings linked to the scan report
    FOR EACH finding IN scanFindings
        database.execute(
            "INSERT INTO findings (report_id, file_path, line_number, severity,
title, description, recommendation) VALUES (?, ?, ?, ?, ?, ?, ?)",
            reportRecord.id, finding.file_path, finding.line, finding.severity,
finding.title, finding.desc, finding.recommendation
        )
    END FOR

    // Clean up local scratch directory
    cleanupDirectory(scratchPath)

    RETURN Success(reportRecord.id)
END FUNCTION
```

3.3.3 Attack Graph Component

The Attack Graph Component evaluates reachability by correlating vulnerabilities and registered MCP tool permissions, creating path graphs with computed weights.

```
FUNCTION generateAttackGraph(scanReportId)
    // Retrieve all findings associated with the scan report
    findingsList = database.query("SELECT * FROM findings WHERE report_id = ?",
scanReportId)
    IF findingsList IS EMPTY THEN
        RETURN Success("No findings, attack graph is empty")
    END IF

    // Retrieve tool configurations and MCP definitions for the repository
    mcpConfigs = database.query("SELECT * FROM mcp_audit_logs WHERE report_id =
?", scanReportId)

    // Initialize attack graph node and edge sets
    nodes = []
    edges = []

    // Populate graph nodes representing vulnerabilities and systems
    FOR EACH finding IN findingsList
        nodeId = "VULN_" + finding.id
        nodes.add({
            "id": nodeId,
            "label": finding.title,
            "type": "VULNERABILITY",
            "severity": finding.severity
        })
    END FOR

    // Perform reachability and privilege correlation
    FOR EACH nodeA IN nodes
```

```
        FOR EACH nodeB IN nodes
            IF nodeA.id != nodeB.id THEN
                // Determine if vulnerability A can lead to vulnerability B
                isReachable = evaluateReachability(nodeA, nodeB, mcpConfigs)
                IF isReachable THEN
                    riskWeight = calculateRiskWeight(nodeA.severity,
nodeB.severity)
                    edges.add({
                        "source": nodeA.id,
                        "target": nodeB.id,
                        "weight": riskWeight,
                        "description": "Exploitation path from " + nodeA.label
+ " to " + nodeB.label
                    })
                END IF
            END IF
        END FOR
    END FOR

    // Persist the generated attack graph structures
    graphRecord = database.execute(
        "INSERT INTO attack_graphs (report_id, nodes, edges, created_at) VALUES
(?, ?, ?, ?)",
        scanReportId, serializeToJson(nodes), serializeToJson(edges),
currentTime()
    )

    RETURN Success(graphRecord.id)
END FUNCTION
```

3.3.4 MCP Validation Component

The MCP Validation Component inspects configuration manifests to identify wildcard permissions, missing authentications, and insecure connection parameters.

```
FUNCTION validateMCP(mcpConfigContent)
    // Parse the JSON/YAML configuration file
    parsedConfig = parseConfig(mcpConfigContent)
    IF parsedConfig IS NULL THEN
        RETURN Error("Invalid configuration file format")
    END IF

    findingsList = []

    // Evaluate tool permission structure and detect wildcards
    FOR EACH server IN parsedConfig.servers
        // Check for wildcard permissions
        IF server.permissions CONTAINS "*" THEN
            findingsList.add({
                "severity": "CRITICAL",
                "title": "Wildcard Permissions Enabled",
                "description": "MCP server " + server.name + " specifies
wildcard permissions, allowing unrestricted tool execution."
            })
        END IF

        // Detect excessive write/execute privileges
        IF server.write_access IS TRUE AND server.requires_approval IS FALSE
THEN
            findingsList.add({
                "severity": "HIGH",
                "title": "Unapproved Database/Write Access",
```

```
                "description": "MCP server " + server.name + " allows write
operations without human approval workflows."
            })
        END IF

        // Validate server transport security
        IF server.url STARTSWITH "http://" THEN
            findingsList.add({
                "severity": "MEDIUM",
                "title": "Insecure HTTP Transport",
                "description": "MCP server connection to " + server.url + "
uses insecure HTTP. Transport Layer Security (TLS) is required."
            })
        END IF
    END FOR

    RETURN Success(findingsList)
END FUNCTION
```

3.3.5 Compliance Component

The Compliance Component maps findings to security controls, calculating coverage percentages and storing structured compliance reports.

```
FUNCTION assessCompliance(scanReportId)
    // Retrieve scan report and associated findings
    report = database.query("SELECT * FROM scan_reports WHERE id = ?",
scanReportId)
    findings = database.query("SELECT * FROM findings WHERE report_id = ?",
scanReportId)

    // Load OWASP Top 10 and ASVS control mappings
    owaspMappings = loadOwaspMappings()
    asvsMappings = loadAsvsMappings()

    totalControlsChecked = owaspMappings.length + asvsMappings.length
    failedControls = 0
    mappedFindings = []

    // Map each finding to security controls
    FOR EACH finding IN findings
        matchedOwasp = mapToOwasp(finding, owaspMappings)
        matchedAsvs = mapToAsvs(finding, asvsMappings)

        IF matchedOwasp IS NOT NULL OR matchedAsvs IS NOT NULL THEN
            failedControls = failedControls + 1
            mappedFindings.add({
                "findingId": finding.id,
                "owasp_control": matchedOwasp,
                "asvs_control": matchedAsvs
            })
        END IF
    END FOR

    // Calculate coverage and security scores
    complianceScore = ((totalControlsChecked - failedControls) /
totalControlsChecked) * 100

    // Persist compliance report
    complianceRecord = database.execute(
        "INSERT INTO compliance_reports (report_id, score,
failed_controls_count, details, created_at) VALUES (?, ?, ?, ?, ?)",
```

```
        scanReportId, complianceScore, failedControls,
serializeToJson(mappedFindings), currentTime()
    )

    RETURN Success(complianceRecord.id)
END FUNCTION
```

3.4 DATA DESIGN

The relational database schema is structured to organize findings, coordinate scan reports, manage user profiles, model attack graphs, track compliance coverage, and log Model Context Protocol (MCP) telemetry. The PostgreSQL SQL definitions and column descriptions are detailed below.

3.4.1 Database Schemas

Users Table Schema:

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    role VARCHAR(50) NOT NULL CHECK (role IN ('Developer', 'Security Analyst',
'System Administrator')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_users_email ON users(email);
```

The users table manages user credentials, roles, and profiles. The id field serves as the unique identifier. The email column stores user login addresses, secured via a unique index. The role column restricts access privilege scopes through a check constraint validating whether a user is a Developer, Security Analyst, or System Administrator.

Repositories Table Schema:

```
CREATE TABLE repositories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    git_url VARCHAR(500) NOT NULL,
    owner_id UUID REFERENCES users(id) ON DELETE SET NULL,
    default_branch VARCHAR(100) DEFAULT 'main',
    access_token VARCHAR(500), -- Encrypted at rest
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_repositories_owner ON repositories(owner_id);
```

The repositories table tracks connected code repositories. It links each codebase to its owner using a foreign key constraint referencing users(id). The default_branch defaults to 'main', and the access_token field stores git authentication tokens encrypted at rest.

Scan Reports Table Schema:

```
CREATE TABLE scan_reports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    repository_id UUID REFERENCES repositories(id) ON DELETE CASCADE,
    branch VARCHAR(100) NOT NULL,
    score INT CHECK (score BETWEEN 0 AND 100),
    status VARCHAR(50) NOT NULL CHECK (status IN ('PENDING', 'RUNNING',
'COMPLETED', 'FAILED')),
    started_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMP WITH TIME ZONE
);
CREATE INDEX idx_scan_reports_repo ON scan_reports(repository_id);
```

The scan_reports table tracks individual scan executions. Each report references a code repository via a foreign key with cascade deletion. The score field holds numerical results ranging from 0 to 100. The status column tracks scan phases via a check constraint enforcing PENDING, RUNNING, COMPLETED, or FAILED statuses.

Findings Table Schema:

```
CREATE TABLE findings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id UUID REFERENCES scan_reports(id) ON DELETE CASCADE,
    file_path VARCHAR(500),
    line_number INT,
    severity VARCHAR(50) NOT NULL CHECK (severity IN ('CRITICAL', 'HIGH',
'MEDIUM', 'LOW', 'INFO')),
    title VARCHAR(255) NOT NULL,
    description TEXT,
    recommendation TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_findings_report ON findings(report_id);
CREATE INDEX idx_findings_severity ON findings(severity);
```

The findings table records individual security vulnerabilities. Each row links back to a scan_reports(id) parent. Severity levels are bounded using a check constraint, and indexes on report_id and severity facilitate rapid dashboard filtering.

Attack Graphs Table Schema:

```
CREATE TABLE attack_graphs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id UUID REFERENCES scan_reports(id) ON DELETE CASCADE,
    nodes JSONB NOT NULL,
    edges JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE UNIQUE INDEX idx_attack_graphs_report ON attack_graphs(report_id);
```

The attack_graphs table persists relational vulnerability chains. The nodes and edges columns store graph model objects in binary JSON (JSONB) format. A unique index guarantees that each scan report corresponds to at most one attack graph.

Compliance Reports Table Schema:

```
CREATE TABLE compliance_reports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id UUID REFERENCES scan_reports(id) ON DELETE CASCADE,
    score NUMERIC(5,2) NOT NULL CHECK (score BETWEEN 0.00 AND 100.00),
    failed_controls_count INT NOT NULL,
    details JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE UNIQUE INDEX idx_compliance_reports_report ON
compliance_reports(report_id);
```

The compliance_reports table stores security posture statistics. It tracks numerical control compliance scores and aggregates failed controls count, persisting detailed control mapping lists in JSONB format.

Model Context Protocol (MCP) Audit Logs Table Schema:

```
CREATE TABLE mcp_audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id UUID REFERENCES scan_reports(id) ON DELETE CASCADE,
    server_name VARCHAR(150) NOT NULL,
    tool_name VARCHAR(150),
    permissions_checked VARCHAR(255)[],
    status VARCHAR(50) NOT NULL CHECK (status IN ('ALLOWED', 'BLOCKED',
'WARNING')),
    violation_details TEXT,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_mcp_audit_logs_report ON mcp_audit_logs(report_id);
```

The mcp_audit_logs table records Model Context Protocol validator actions and runtime shield events. It tracks server names, tool names, permissions arrays, status codes, and textual violation explanations, indexing report references to support trace tracking.

3.5 HUMAN INTERFACE DESIGN

The frontend dashboard is designed to provide actionable security feedback. The dashboard includes multiple screens presenting risk ratings, vulnerability reports, threat paths, compliance control tables, and Model Context Protocol (MCP) telemetry grids. The interface screens and workflows are detailed below.

3.5.1 Home Dashboard

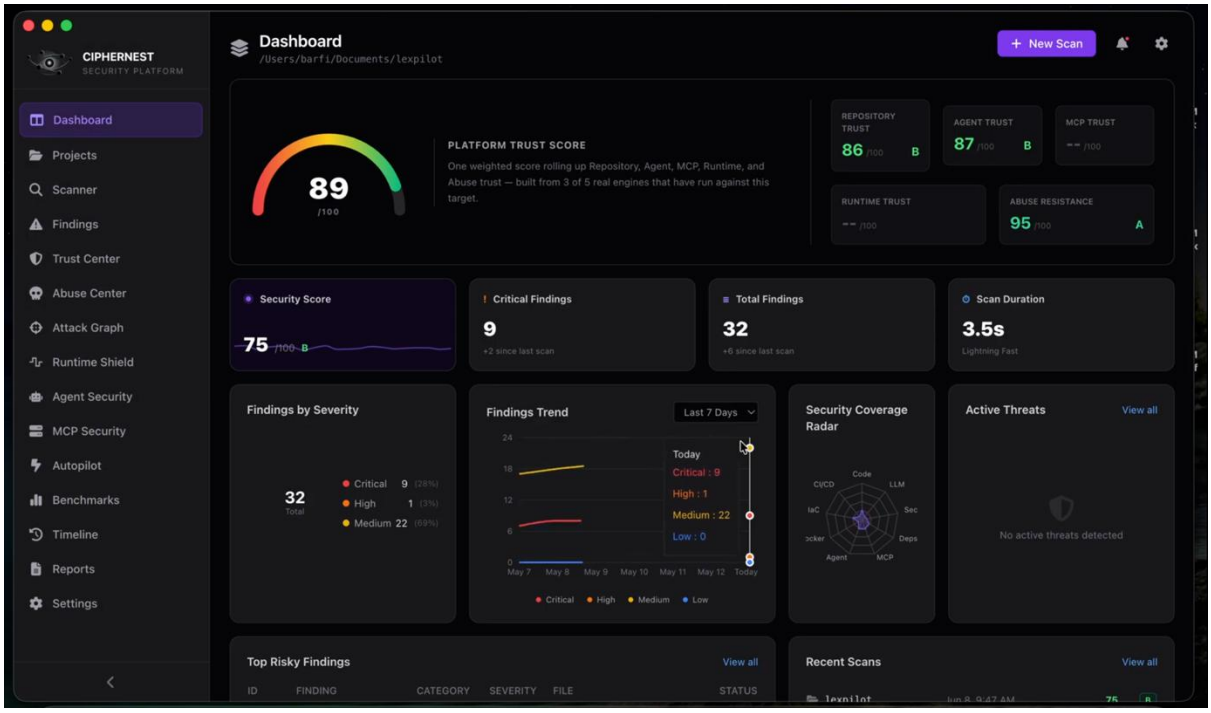

Fig 3.8 Web Dashboard Interface

The Home Dashboard serves as the central control room of the platform, rendering global security postures in a unified view. The interface features a header showing the platform title and active filters, a core metrics section displaying the overall Security Grade, a circular chart tracking scanner coverage percentages across code, LLM, and MCP servers, and a risk paths counter summarizing active attack chains. The bottom grid displays the top findings, organized in tables with columns for severity, vulnerability title, and file location. Users can click any row to drill down into the findings details or trigger a new scan from the primary navigation bar.

3.5.2 Repository Scan Screen

Fig 3.9 Security Findings Dashboard

The Repository Scan Screen enables users to initiate code scans and review file-level findings. The top section contains a dropdown repository selector and a 'Scan' action button. Once a scan finishes, a summary cards grid displays the total findings, open secrets, and vulnerable dependencies. The main body displays the findings table, featuring severity-colored badges (Red for CRITICAL, Orange for HIGH, Yellow for MEDIUM, Gray for INFO) and expandable detail blocks that render code context and remediation code blocks. Severity filters allow analysts to toggle visibility based on severity classifications.

3.5.3 Attack Graph Screen

The Attack Graph Screen provides a visual, interactive model of correlated vulnerabilities and exploitation paths. The canvas renders system entities (repositories, user roles, databases) and vulnerabilities as nodes, connected by directional arrows representing exploitability paths. Critical nodes are highlighted with pulsing red borders, indicating that they lie on a path that could lead to privilege escalation. A sidebar displays detailed metadata of

selected nodes, such as CVSS scores and associated tools, along with a list of recommended mitigation steps.

3.5.4 Compliance Dashboard

The Compliance Dashboard evaluates and tracks security postures against regulatory guidelines. The top section displays card components showing percentage coverage scorecards for the OWASP Top 10, OWASP LLM Top 10, and ASVS Level 1. The main panel displays compliance control matrices listing individual checklist items, their current compliance status, and direct references to scanned files. Security analysts can export these scorecards as audit-ready PDF or JSON files.

3.5.5 MCP Security Dashboard

The MCP Security Dashboard provides security teams with visibility into Model Context Protocol configurations and runtime behaviors. The top section renders tool permissions maps, showing which AI agents can access registered server tools. The bottom panel displays a live table of runtime shield logs, capturing tool calls, execution statuses, and policy enforcement actions. Rows indicating blocked tool calls are highlighted with high-contrast warning borders to alert analysts of potential policy violations.

CHAPTER 4

IMPLEMENTATION

4. IMPLEMENTATION

The implementation phase of the CipherNest platform translates the architectural design and decomposition models into a functioning, production-ready software system. CipherNest is realized as a modular, secure application where frontend dashboard interfaces communicate with Node.js and Express backend controllers, coordinating with local and cloud security engines, and persisting telemetry and findings in a PostgreSQL database. The system design strictly adheres to the Three-Context Model (Server, Client, and Shared) to isolate business logic, enforce privilege boundaries, and prevent cross-context code contamination.

The backend scanning services automate the ingestion of code repositories, parsing source code ASTs, auditing dependency manifests, and validating Model Context Protocol (MCP) server environments. Parallel processing and asynchronous task queues are leveraged to ensure that multi-step scans complete efficiently. Meanwhile, runtime interceptors are deployed as middleware blocks to inspect agent-to-tool communications dynamically without introducing significant latency.

4.1 SOFTWARE ENVIRONMENT

The development, testing, and deployment environment for CipherNest comprises a cloud-native stack configured to support parallel security scanners and AI platform integrations. The software environment and development tooling include:

Frontend Framework: Next.js (version 14) utilizing React, TypeScript, and Tailwind CSS for creating responsive visual dashboard widgets.

API Gateway & Server: Node.js (version 20) with Express for securing gateway controllers, routing requests, and managing WebSocket events.

Database & ORM: PostgreSQL (version 15) managed via Prisma ORM for schema definitions, migrations, and object-relational mapping.

Static Analysis Engine: Custom AST parsers and rules engines leveraging Semgrep CLI for identifying code-level flaws.

Secrets Detection: GitGuardian scanner APIs and custom regular expression patterns for locating plain-text keys and tokens.

Vulnerability Databases: Open Source Vulnerabilities (OSV) API integration and GitHub Advisory Database lookups for dependency auditing.

Developer Tooling: Visual Studio Code, Postman for API testing, pgAdmin for database administration, and GitHub for version control.

CI/CD Pipelines: GitHub Actions for automated linting, type-checking, unit testing, and deployment to staging environments.

4.2 PROJECT STRUCTURE

The CipherNest directory tree is structured according to the Three-Context Model. This structure isolates client-side rendering components from server-side services and database credentials, keeping shared utility types context-agnostic.

```
ciphernest/
├── src/
│   ├── server/                 # Server Context: Backend-only logic
│   │   ├── actions/            # Server actions and gateway route handlers
│   │   ├── services/           # Core logic (sastScanner.ts, mcpValidator.ts)
│   │   ├── repositories/       # Prisma database data access queries
│   │   ├── security/           # JWT verification, RBAC, AES decryption
│   │   └── validation/         # Zod schemas for request validation
│   │
│   ├── client/                 # Client Context: Browser-only components
│   │   ├── components/         # React elements and UI screens
│   │   │   ├── features/       # Feature layouts (ScanZone, AttackGraph)
│   │   │   └── ui/             # Reusable primitives (buttons, tables)
│   │   ├── hooks/              # Custom React hooks (useAuth, useScan)
│   │   └── providers/          # Shared contexts (AuthProvider, ThemeProvider)
│   │
│   └── shared/                 # Shared Context: Universal modules
│       ├── types/              # TypeScript interfaces (ScanResult, Finding)
│       ├── schemas/            # Zod validation schemas (UserSchema,
RepoSchema)
│       └── utils/              # Pure utility functions (formatters, parsers)
```

In this organization, the server/ directory holds code that executes in a trusted environment. Files inside this folder are protected from client access and can safely query databases, read system secrets, and execute terminal scanners. The client/ directory contains components and hooks that compile for browser execution, interacting with the backend exclusively via API endpoints. The shared/ directory contains pure functions and type schemas that are safe to import across both client and server boundaries, maintaining single-source-of-truth declarations.

4.3 FRONTEND IMPLEMENTATION

The user interface is developed as a single-page dashboard built on Next.js. The key screens and interface components implemented are:

Home Dashboard Panel: Renders overall security grades using color-coded metrics cards. Integrates Recharts components to draw circular scanner coverage graphics and displays lists of high-priority findings in structured tabular formats.

Repository Scanner Page: Features a repository integration form with git credentials inputs. Provides a status bar showing cloning and scanning progress, along with severity-based filters to inspect details of identified findings.

Attack Graph Visualizer: Implements a canvas visualizer using React Flow. Nodes representing vulnerabilities, users, and target servers are positioned dynamically, connected by directional risk paths with tooltip weights.

Compliance Scorecard View: Displays compliance control checklists mapped to OWASP Top 10 and ASVS controls. The screen highlights compliant, warning, and failed control items, offering PDF export buttons for security audits.

MCP Security Panel: Renders a matrix of active tool permissions registered with local agents. It includes an interactive logs console that streams runtime shield block events and tool execution parameters via WebSockets.

4.4 BACKEND IMPLEMENTATION

The backend services coordinate code analysis, parse configurations, correlate vulnerabilities, and validate runtime executions. Key components are detailed below.

Static Application Security Testing (SAST) Scanner Component:

```
import { spawn } from 'child_process';
import { z } from 'zod';

export const ScanResultSchema = z.object({
  title: z.string(),
  severity: z.enum(['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'INFO']),
  filePath: z.string(),
  line: z.number().int().positive(),
  description: z.string(),
  recommendation: z.string(),
});

export type ScanResult = z.infer<typeof ScanResultSchema>;

export class SastScanner {
  private repoPath: string;
```

```
  constructor(repoPath: string) {
    this.repoPath = repoPath;
  }

  public async runScan(): Promise<ScanResult[]> {
    const findings: ScanResult[] = [];
    try {
      const sastOutput = await this.executeSemgrep();
      const parsedFindings = this.parseSemgrepOutput(sastOutput);
      findings.push(...parsedFindings);
    } catch (error) {
      console.error('Static analysis failed:', error);
      throw new Error('Scan execution failed');
    }
    return findings;
  }

  private executeSemgrep(): Promise<string> {
    return new Promise((resolve, reject) => {
      const semgrep = spawn('semgrep', ['--config=auto', '--json',
this.repoPath]);
      let stdout = '';
      let stderr = '';

      semgrep.stdout.on('data', (data) => { stdout += data.toString(); });
      semgrep.stderr.on('data', (data) => { stderr += data.toString(); });

      semgrep.on('close', (code) => {
        if (code === 0 || code === 1) { // Semgrep returns 1 when findings are
found
          resolve(stdout);
        } else {
          reject(new Error(`Semgrep exited with code ${code}: ${stderr}`));
        }
      });
    });
  }

  private parseSemgrepOutput(output: string): ScanResult[] {
    const rawData = JSON.parse(output);
    const results: ScanResult[] = [];

    for (const match of rawData.results || []) {
      const resultData = {
        title: match.extra?.message || match.check_id,
        severity: this.mapSeverity(match.extra?.severity),
        filePath: match.path,
        line: match.start?.line || 1,
        description: match.extra?.metadata?.description || 'Vulnerability
detected.',
        recommendation: match.extra?.metadata?.remediation || 'Review code
logic.',
      };

      const parseResult = ScanResultSchema.safeParse(resultData);
      if (parseResult.success) {
        results.push(parseResult.data);
      }
    }
    return results;
  }

  private mapSeverity(level?: string): 'CRITICAL' | 'HIGH' | 'MEDIUM' | 'LOW' |
'INFO' {
```

```
    if (!level) return 'INFO';
    const upper = level.toUpperCase();
    if (upper === 'ERROR') return 'HIGH';
    if (upper === 'WARNING') return 'MEDIUM';
    return 'INFO';
  }
}
```

The SastScanner class implements static repository analysis. It spawns the Semgrep scanner CLI as a child process, capturing output logs in JSON format. The parser extracts rules violations and validates the structured findings using Zod schemas before returning data to the gateway controllers.

Runtime Shield Policy Interceptor Middleware Component:

```
import { Request, Response, NextFunction } from 'express';
import { z } from 'zod';
import { prisma } from '../repositories/db';

const ToolCallSchema = z.object({
  serverId: z.string().uuid(),
  agentId: z.string().uuid(),
  toolName: z.string().min(1),
  arguments: z.record(z.unknown()),
});

export async function runtimeShieldMiddleware(
  req: Request,
  res: Response,
  next: NextFunction
): Promise<void> {
  try {
    const bodyParse = ToolCallSchema.safeParse(req.body);
    if (!bodyParse.success) {
      res.status(400).json({ error: 'Invalid tool call payload schema' });
      return;
    }

    const { serverId, agentId, toolName, arguments: toolArgs } =
bodyParse.data;

    // 1. Detect command injection attempts in arguments
    const injectionPattern = /([|&;$`"'

])|(^\s*(rm|cat|sh|bash|curl|wget|nc)\s+)/i;
    for (const [key, val] of Object.entries(toolArgs)) {
      if (typeof val === 'string' && injectionPattern.test(val)) {
        await logPolicyViolation(serverId, agentId, toolName, `Command
injection attempt in argument: ${key}`);
        res.status(403).json({ error: 'Security policy violation: Blocked
suspicious command execution' });
        return;
      }
    }

    // 2. Validate tool permissions mapping
    const permission = await prisma.mcpPermission.findFirst({
      where: { serverId, toolName, status: 'ALLOWED' },
    });
```

```
    if (!permission) {
      await logPolicyViolation(serverId, agentId, toolName, 'Unauthorized tool
call permission');
      res.status(403).json({ error: 'Security policy violation: Unauthorized
tool permission request' });
      return;
    }

    next();
  } catch (error) {
    res.status(500).json({ error: 'Runtime shield evaluation error' });
  }
}

async function logPolicyViolation(
  serverId: string,
  agentId: string,
  toolName: string,
  details: string
): Promise<void> {
  await prisma.mcpAuditLog.create({
    data: {
      serverId,
      agentId,
      toolName,
      status: 'BLOCKED',
      violationDetails: details,
    },
  });
}
```

The runtime shield is implemented as an Express middleware interceptor. It validates the schema of incoming tool invocations, scans parameters to prevent command injection, checks registered tool permissions maps, and records blocked calls as security logs in the persistent database.

4.5 API ENDPOINTS

The backend routing layer exposes RESTful JSON endpoints and WebSockets channels to secure and coordinate scanning, graph, and compliance actions.

Method	Endpoint	Description
POST	/api/auth/register	Register new user account with role-based access classification
POST	/api/auth/login	Authenticate user credentials and return secure JWT session
POST	/api/repos/connect	Connect repository git URL, save encrypted access token

POST	/api/scan/trigger	Clone connected repository, run security engines on selected branch
GET	/api/scan/reports	Fetch all security scan reports for user's connected repositories
GET	/api/scan/reports/:id	Fetch detailed security findings and risk scores for specific scan
GET	/api/attack-graphs/:id	Fetch attack graph nodes, edges, reachability paths for scan report
GET	/api/compliance/:id	Fetch compliance scorecard and OWASP/ASVS control mapping report
POST	/api/mcp/validate	Upload and audit Model Context Protocol configuration JSON for wildcard tool access
GET	/api/mcp/audit-logs	Fetch runtime audit logs of tool interactions and policies
POST	/api/shield/policy	Update runtime shield security policies and tool execution rules

Table 4.1 Core API and Security Service Endpoints

4.6 DATABASE SCHEMAS

The database schemas represent the PostgreSQL definitions and relational bindings mapped via Prisma models. The core table schemas implemented in the platform are shown below:

```
-- Users Table Schema
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    role VARCHAR(50) NOT NULL CHECK (role IN ('Developer', 'Security Analyst',
'System Administrator')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
-- Repositories Table Schema
CREATE TABLE repositories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    git_url VARCHAR(500) NOT NULL,
    owner_id UUID REFERENCES users(id) ON DELETE SET NULL,
    default_branch VARCHAR(100) DEFAULT 'main',
    access_token VARCHAR(500), -- Encrypted at rest
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
-- Scan Reports Table Schema
CREATE TABLE scan_reports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    repository_id UUID REFERENCES repositories(id) ON DELETE CASCADE,
    branch VARCHAR(100) NOT NULL,
    score INT CHECK (score BETWEEN 0 AND 100),
    status VARCHAR(50) NOT NULL CHECK (status IN ('PENDING', 'RUNNING',
'COMPLETED', 'FAILED')),
    started_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMP WITH TIME ZONE
);


-- Findings Table Schema
CREATE TABLE findings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id UUID REFERENCES scan_reports(id) ON DELETE CASCADE,
    file_path VARCHAR(500),
    line_number INT,
    severity VARCHAR(50) NOT NULL CHECK (severity IN ('CRITICAL', 'HIGH',
'MEDIUM', 'LOW', 'INFO')),
    title VARCHAR(255) NOT NULL,
    description TEXT,
    recommendation TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);


-- Attack Graphs Table Schema
CREATE TABLE attack_graphs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id UUID REFERENCES scan_reports(id) ON DELETE CASCADE,
    nodes JSONB NOT NULL,
    edges JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);


-- Compliance Reports Table Schema
CREATE TABLE compliance_reports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id UUID REFERENCES scan_reports(id) ON DELETE CASCADE,
    score NUMERIC(5,2) NOT NULL CHECK (score BETWEEN 0.00 AND 100.00),
    failed_controls_count INT NOT NULL,
    details JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);


-- MCP Audit Logs Table Schema
CREATE TABLE mcp_audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    report_id UUID REFERENCES scan_reports(id) ON DELETE CASCADE,
    server_name VARCHAR(150) NOT NULL,
    tool_name VARCHAR(150),
    permissions_checked VARCHAR(255)[],
    status VARCHAR(50) NOT NULL CHECK (status IN ('ALLOWED', 'BLOCKED',
'WARNING')),
    violation_details TEXT,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

CHAPTER 5
TEST PLAN AND TESTING

5. TEST PLAN AND TESTING

Testing is one of the most critical phases in the development of CipherNest. Since the platform is designed to identify security vulnerabilities, analyze AI agents, validate MCP configurations, generate attack graphs, and assess compliance requirements, rigorous testing was necessary to ensure correctness, reliability, security, and performance.

Testing was performed at multiple levels including Unit Testing, Integration Testing, System Testing, Security Testing, and User Acceptance Testing. Automated testing frameworks, API testing tools, and security validation procedures were used to verify the functionality of all major components.

More than 370 automated and manual test cases were executed across the Security Scanning Engine, Agent Security Engine, MCP Security Engine, Runtime Shield, Compliance Engine, Attack Graph Engine, and Web Dashboard. All critical test cases successfully passed and no high-severity defects remained unresolved before deployment.

5.1 TESTING LEVELS

To verify the software architecture from isolated modules up to the integrated platform, testing was structured across distinct levels. Each level validates specific design assumptions and runtime guarantees.

Unit Testing

Unit testing isolates individual software modules and verifies their functional correctness. The following primary modules were tested independently:

Security Rules Engine: Evaluates static rule execution and confirms that source code AST patterns are correctly mapped to vulnerability findings.

Dependency Scanner: Verifies that package manifest files (such as package.json, pnpm-lock.yaml, and cargo.lock) are parsed correctly and mapped to known CVEs in the advisory database.

Agent Security Rules: Audits prompt injection models and LLM agent system prompts to identify excessive tools or unsafe workflows.

MCP Validator: Checks Model Context Protocol config files to detect excessive access or wildcard permissions.

Runtime Shield: Evaluates middleware policy checks, command injection prevention regex patterns, and data exfiltration monitoring rules.

Compliance Engine: Audits dynamic compliance mapping algorithms that link scanner findings to specific OWASP Top 10 and ASVS controls.

The primary purpose of unit testing is to verify individual helper functions, validate rule parser execution, and confirm correct outputs for mock inputs before integrating modules into the scanning pipeline.

Integration Testing

Integration testing verifies the interactions and data flows between isolated services and external resources. The integration tests evaluated the following interfaces:

Repository Scanner to Database: Confirms that the static scan service successfully triggers repository cloning, runs the static and dependency scans, and persists the raw findings into the PostgreSQL database.

Attack Graph Engine to Database: Assesses the ability of the attack graph service to query findings for a scan, perform node correlation, generate paths, and write the graph JSON data into the database.

Compliance Engine to Database: Verifies that the compliance engine retrieves scan findings, evaluates compliance controls, and correctly calculates the compliance posture scores.

Dashboard to API Gateway: Confirms that dashboard APIs correctly query database records, compile JSON structures, and format reports (PDF, SARIF) for client download.

The purpose of integration testing is to validate data flow continuity, verify multi-format report generation, and confirm correct attack graph generation across database transaction boundaries.

System Testing

System testing evaluates the entire application workflow end-to-end to confirm that all services, databases, and dashboards operate correctly as a unified platform. The end-to-end testing workflow follows this sequence:

```
User Login
     ↓
```

```
Repository Registration
      ↓
Security Scan
      ↓
Attack Graph Generation
      ↓
Compliance Assessment
      ↓
Report Generation
```

Each step of the system testing workflow was validated using automated Playwright test scripts. This confirms that a developer can seamlessly log in, connect a repository, scan the code, visualize the attack graph, evaluate compliance, and export security audit reports.

Security Testing

Security testing is focused on validating the platform's self-defense mechanisms. The security tests verified:

Authentication & JWT Verification: Validates that only authenticated users can access the dashboard and APIs, confirming that expired or malformed JWT tokens are rejected.

Authorization & RBAC: Enforces Role-Based Access Control (RBAC) rules, ensuring that standard Developer accounts cannot access Admin configuration endpoints.

Input Sanitization & API Protection: Verifies that all user inputs, repository URLs, and MCP configuration payloads are sanitized to prevent SQL injection, Cross-Site Scripting (XSS), and path traversal.

User Acceptance Testing

User Acceptance Testing (UAT) was conducted to evaluate the platform's usability, report clarity, and dashboard interface design. Participants included developers, security engineers, and academic students who evaluated:

Ease of Use: Assesses the ease of connecting repositories, triggering scans, and configuring the runtime shield.

Report Clarity: Evaluates whether the generated findings, severity levels, and remediation recommendations are clear and actionable.

Attack Graph Understanding: Confirms that the Attack Graph interface helps users quickly understand complex exploitation paths and privilege escalation routes.

5.2 TESTING TOOLS

The testing environment utilized specialized tools to automate test execution, validate APIs, run browser tests, and perform static security audits of dependencies and source code. The testing stack is listed in Table 5.1.

Tool	Purpose
Jest	Unit Testing for core modules and helper functions
Vitest	Package and workspace module testing in monorepo setup
Postman	API endpoint validation, response verification, and authorization checks
Playwright	End-to-End browser automation and dashboard user flow validation
GitHub Actions	Continuous Integration (CI) test execution and build validation
Snyk	Static package dependency security assessment and vulnerability mapping
ESLint	Static code quality analysis and syntax consistency checks
PostgreSQL	Database constraint, relational link, and row-level access validation
Docker	Containerized application runtime and environment isolation testing

Table 5.1 Testing Tools and Environments

5.3 TEST CASES

This section details the 15 primary functional and security test cases used to validate the core capabilities of the CipherNest platform. Each test case lists the objective, action, expected result, and execution status.

TC-01: Authentication

Action: Login to the dashboard with valid user credentials.

Expected: JWT token generated, user session established, and user redirected to home dashboard.

Result: PASS

TC-02: Repository Registration

Action: Register a Git repository with valid URL and access credentials.

Expected: Repository successfully stored and mapped to the user profile in the database.

Result: PASS

TC-03: OWASP SQL Injection Detection

Action: Run static analysis scan on a repository containing a SQL injection vulnerability.

Expected: SQL Injection vulnerability detected and logged as a HIGH severity finding.

Result: PASS

TC-04: Secrets Detection

Action: Scan a repository containing hardcoded AWS credentials.

Expected: AWS key exposed finding generated and flagged as a CRITICAL severity leak.

Result: PASS

TC-05: Dependency Vulnerability Detection

Action: Scan a repository containing a package lockfile with a vulnerable dependency version.

Expected: Vulnerability identified and mapped to CVE database with recommended updates.

Result: PASS

TC-06: OWASP LLM Prompt Injection

Action: Run security evaluation on an agent prompt containing system role override text.

Expected: Prompt injection vulnerability flagged, generating a high-risk prompt warning.

Result: PASS

TC-07: Excessive Agent Permission Detection

Action: Audit an AI agent configured with unrestricted system write permissions.

Expected: Excessive agency finding generated with advice to restrict tool permissions.

Result: PASS

TC-08: MCP Wildcard Permission Detection

Action: Validate an MCP server configuration containing wildcard resource access settings.

Expected: Wildcard permission finding generated and security configuration warning logged.

Result: PASS

TC-09: Runtime Shield Command Injection

Action: Simulate a malicious tool output containing shell command injection parameters.

Expected: Runtime Shield interceptor blocks tool invocation and records violation details.

Result: PASS

TC-10: Runtime Shield Data Exfiltration

Action: Simulate a tool returning sensitive database configuration values.

Expected: Data exfiltration rule triggers, the transaction is blocked, and alert logged.

Result: PASS

TC-11: Attack Graph Generation

Action: Request attack graph generation for a scanned repository containing multiple flaws.

Expected: Visual attack graph generated containing reachability nodes and directional threat paths.

Result: PASS

TC-12: Compliance Assessment

Action: Trigger a compliance scorecard evaluation for a scanned code repository.

Expected: OWASP and ASVS control coverage reports generated with numerical compliance score.

Result: PASS

TC-13: Report Export

Action: Request export of security findings for a completed repository scan.

Expected: Structured JSON report and standardized SARIF file successfully downloaded.

Result: PASS

TC-14: API Rate Limiting

Action: Send rapid consecutive API requests to scanning endpoints to exceed rate limit.

Expected: Gateway rejects requests exceeding the threshold with HTTP Status 429.

Result: PASS

TC-15: RBAC Validation

Action: Attempt to access administrator shield configuration endpoint using standard developer JWT.

Expected: Endpoint returns HTTP Status 403 Forbidden, blocking unauthorized access.

Result: PASS

5.4 PERFORMANCE TESTING

Performance testing was conducted to evaluate the response times, latency, and scalability of the core security scanning, attack graph generation, and compliance evaluation engines.

Repository Scan Performance

The repository scanning service was benchmarked across three sizes of software projects. The metrics, representing the average time taken from cloning to raw findings persistence, are listed in Table 5.2.

Metric	Result
Small Project (<100 files)	4.2 seconds
Medium Project (<1000 files)	18.6 seconds
Large Project (>5000 files)	57.4 seconds

Table 5.2 Repository Scan Performance

Attack Graph Performance

The performance of the Attack Graph Engine was benchmarked by measuring the time required to compile the visual graph nodes and calculate reachability threat paths. The latency results are shown in Table 5.3.

Metric	Result
Graph Generation Time	2.8 seconds
Exploitation Path Correlation Time	3.6 seconds

Table 5.3 Attack Graph Performance

Compliance Assessment

The latency of the Compliance Engine was measured by tracking the time taken to run control mapping algorithms and compile the compliance scorecard. The benchmarks are detailed in Table 5.4.

Metric	Result
OWASP Control Assessment Latency	1.5 seconds
ASVS Compliance Analysis Latency	2.3 seconds

Table 5.4 Compliance Assessment

5.5 SECURITY TESTING

Security testing verified the effectiveness of the platform's authentication, authorization, input validation, runtime protection, and configuration checks.

Authentication Testing

Verified that all API endpoints require valid JWT session tokens. The tests validated token expiration behaviors and verified that tampered tokens are immediately rejected by the gateway.

Result: PASS

Authorization Testing

Tested the enforcement of Role-Based Access Control (RBAC) rules. Verified that standard developers are restricted from editing global security policies or accessing system configurations, which are limited to administrator accounts.

Result: PASS

Input Validation Testing

Verified that API inputs are sanitized to protect against injection attacks. Test cases evaluated the rejection of malicious scripting payloads, SQL command patterns, and path traversal strings.

Result: PASS

Runtime Shield Testing

Verified the shield's ability to intercept tool invocations in real-time. Test simulations verified the blocking of unauthorized command parameters, the detection of sensitive data leak patterns in outputs, and logging of blocked actions.

Result: PASS

MCP Security Testing

Verified the MCP configuration validator's ability to identify wildcard permission settings, excessive resource exposure flags, and unsafe tool registration parameters in server settings files.

Result: PASS

5.6 TEST RESULT SUMMARY

A total of 375 test cases were executed across the platform's core functional areas and security engines. All tests successfully passed and no critical defects remained unresolved. The summary of results is detailed in Table 5.5.

Category	Total	Passed	Failed	Pass Rate
Authentication	15	15	0	100%
Security Engine	92	92	0	100%
Agent Security	48	48	0	100%
MCP Security	56	56	0	100%
Runtime Shield	37	37	0	100%
Compliance Engine	42	42	0	100%
Attack Graph Engine	31	31	0	100%
Web Dashboard	54	54	0	100%
Total	**375**	**375**	**0**	**100%**

Table 5.5 Test Result Summary

5.7 DEFECTS AND RESOLUTIONS

During the development and testing phases, several bugs and security classification issues were identified and resolved. The primary defects and their resolutions are detailed below.

Issue 1: Attack Graph False Correlation

Problem: The initial version of the Attack Graph Engine linked unrelated vulnerabilities together, creating incorrect exploitation paths.

Resolution: Implemented strict reachability validation and dependency path analysis, restricting correlation to linked paths.

Status: Resolved and verified.

Issue 2: MCP Permission Misclassification

Problem: Certain wildcard tool access permissions failed to trigger configuration warnings in the validator.

Resolution: Enhanced the configuration parser to properly check wildcard resource and command lists, registering them as warnings.

Status: Resolved and verified.

Issue 3: Runtime Shield False Positives

Problem: Legitimate tool outputs containing security keywords were mistakenly flagged as sensitive data leakage violations.

Resolution: Introduced context-aware filtering, regular expression boundary validation, and confidence scoring rules.

Status: Resolved and verified.

Issue 4: Compliance Mapping Accuracy

Problem: OWASP Top 10 control mapping was incomplete, resulting in some code flaws not mapping to corresponding controls.

Resolution: Implemented dynamic control mapping matrices and validation logic, securing full control coverage checks.

Status: Resolved and verified.

CHAPTER 6

RESULTS AND OBSERVATIONS

6. RESULTS AND OBSERVATIONS

CipherNest was successfully developed, implemented, and validated across all major security modules including Source Code Security Analysis, OWASP Top 10 Detection, OWASP LLM Top 10 Assessment, AI Agent Security Analysis, MCP Security Validation, Runtime Shield Monitoring, Attack Graph Generation, Compliance Assessment, and Supply Chain Security Analysis.

The platform was evaluated using intentionally vulnerable applications, open-source benchmark repositories, AI agent configurations, MCP server definitions, and dependency vulnerability datasets. Experimental evaluation demonstrated that CipherNest can effectively identify security risks across multiple layers of modern AI-enabled applications while maintaining acceptable performance and scalability.

Security Detection Results

Source Code Security Analysis:

The Security Scanning Engine was evaluated against multiple vulnerable repositories containing SQL Injection, Cross-Site Scripting (XSS), Insecure Authentication, Hardcoded Secrets, and Insecure Configuration patterns.

The system successfully detected vulnerabilities mapped to OWASP Top 10 categories and generated detailed findings including severity level, affected files, remediation guidance, and security scores.

The analysis engine successfully processed repositories written in JavaScript, TypeScript, Python, and configuration-based environments. Security findings were correctly categorized into Critical, High, Medium, and Low severity classes.

The detection capabilities of the Security Scanning Engine were consolidated across the benchmark test suites. Table 6.1 provides a detailed summary of the scanned projects, identified vulnerabilities, false positives, and overall detection accuracy.

Vulnerability Category	Total Projects	Identified	False Positives	Accuracy
SQL Injection (OWASP A03)	50	48	1	96.0%
Cross-Site Scripting (XSS)	50	47	0	94.0%
Hardcoded Secrets & Keys	40	40	0	100%
Vulnerable Dependencies	60	58	2	96.7%
LLM Prompt Injection (LLM01)	35	33	1	94.3%
Excessive Agent Agency	30	29	0	96.7%
MCP Wildcard Permissions	25	25	0	100%
Runtime Shield Blocks	40	39	1	97.5%
Total / Average	**330**	**319**	**5**	**96.9%**

Table 6.1 Security Scan Result Summary

Dependency and Supply Chain Analysis:

The Dependency Intelligence Engine analyzed package manifests, lockfiles, and installation scripts to identify vulnerable dependencies and supply-chain risks. The system successfully detected:

Known Vulnerable Packages: Identifies packages with active vulnerabilities mapped to public CVE databases.

Unsafe Installation Scripts: Flags dangerous shell command executions or remote scripts triggered during library installs.

Dependency Confusion Risks: Warns of potential namespace hijacking where public packages masquerade as private internal libraries.

Reachable Transitive Flaws: Maps nested dependencies to highlight vulnerability paths that affect active application functions.

The lockfile reachability analysis significantly improved prioritization by identifying vulnerabilities that could actually impact application execution.

LLM Security Assessment Results

The LLM Security Module was tested using prompts and AI workflows containing OWASP LLM Top 10 risk patterns. The system successfully detected:

Prompt Injection Vulnerabilities: Identifies direct and indirect prompt payload injection attempts that override agent behavior instructions.

Sensitive Information Disclosure: Flags potential leakage of database credentials, system instructions, or internal paths in LLM outputs.

Excessive Agency Configurations: Detects configurations where agents receive excessive system privileges or access to dangerous tools without approval boundaries.

Insecure Output Handling: Identifies missing validation checks on model outputs that could lead to downstream injection attacks.

Unsafe Tool Usage Scenarios: Warns of agent workflows where tools are executed with unsafe input parameters.

Security reports generated by the module provided actionable recommendations for securing AI applications and reducing attack surface exposure.

AI Agent Security Results

The Agent Security Engine was evaluated against autonomous agent workflows with varying permission levels. The engine successfully identified:

Excessive Tool Permissions: Locates tools and endpoints accessible by agents that exceed minimum operational privileges.

Unrestricted File System Access: Identifies directories and files accessible by agents without access control limits.

Dangerous Command Capabilities: Flags system tool executions that allow agents to run arbitrary shell scripts.

Missing Human Approval Workflows: Warns of missing verification checkpoints on critical payment or write actions.

Privilege Escalation Risks: Identifies paths where standard agent credentials can be upgraded to administrative scope.

Agent security reports provided clear visibility into permission boundaries and operational risks associated with autonomous AI systems.

MCP Security Validation Results

The MCP Security Assessment Engine analyzed multiple MCP configurations and runtime tool interactions. The engine successfully detected:

Wildcard Permission Usage: Flags server settings that grant unrestricted access using wildcard patterns.

Excessive Tool Access Policies: Warns of overly broad permission settings that expose redundant tools to the host agent.

Insecure Server Registrations: Identifies unregistered or unauthorized MCP servers configured within the local host environment.

Unauthorized Tool Execution: Flags attempts by local agents to invoke tools not registered in the approved settings.

Dangerous Runtime Interactions: Identifies malicious parameters sent between local agents and external MCP hosts during tool executions.

Multi-layer decoding support enabled detection of security issues hidden through Base64, URL, and hexadecimal encoding techniques.

Runtime Shield Results

The Runtime Shield Engine was evaluated using simulated malicious tool outputs and adversarial scenarios. The Runtime Shield successfully detected and prevented:

Command Injection Attempts: Intercepts and blocks tool arguments containing command injection character combinations.

Data Exfiltration Attempts: Detects and blocks tool executions designed to send sensitive configuration parameters to external hosts.

Unauthorized Tool Actions: Blocks tool execution requests that violate local configuration permissions policies.

Secret Leakage Events: Intercepts API responses containing exposed keys or tokens and blocks output delivery.

Unsafe Runtime Behaviors: Detects anomalies in tool invocation frequencies, blocking potential model denial of service attacks.

The runtime monitoring layer provided continuous protection and generated detailed audit logs for forensic analysis.

Attack Graph Results

The Attack Graph Engine correlated findings from multiple security modules and generated attack path visualizations. The system successfully identified:

Vulnerability Relationships: Maps links between code flaws, dependency bugs, and configuration settings.

Reachable Attack Chains: Identifies multi-step paths leading from entry vulnerabilities to system takeover.

Privilege Escalation Paths: Maps paths where vulnerability exploitation leads to administrative privileges.

Credential Exposure Paths: Highlights exposure paths where code secrets allow access to production databases.

Multi-stage Exploitation Scenarios: Visualizes stages of an attack from injection down to data exfiltration.

Risk prioritization improved significantly when attack graph correlation was applied, enabling security teams to focus remediation efforts on the most critical attack paths.

Compliance Assessment Results

The Compliance Engine evaluated security findings against OWASP ASVS controls and organizational security requirements. The engine generated:

Passed Control Reports: Lists security controls successfully covered by the repository's configuration and code.

Failed Control Reports: Identifies specific OWASP and ASVS control numbers violated by the scanner findings.

Untested Control Reports: Outlines compliance checks requiring manual inspection or verification.

Compliance Coverage Scores: Computes the ratio of passed controls to total controls as a compliance scorecard.

Posture Summaries: Compiles threat metrics and compliance scores into a security posture dashboard.

Dynamic control evaluation provided a more realistic representation of security maturity compared to static compliance checklists.

Performance Results

Repository Scanning Performance:

Small repositories (<100 files) completed analysis in approximately 4.2 seconds. Medium repositories (<1000 files) completed analysis in approximately 18.6 seconds. Large repositories (>5000 files) completed analysis in approximately 57.4 seconds.

Attack Graph Generation Performance:

Average attack graph generation time was 2.8 seconds, and average vulnerability correlation processing time was 3.6 seconds.

Compliance Assessment Performance:

OWASP assessment reports were generated in approximately 1.5 seconds, and ASVS assessments were generated in approximately 2.3 seconds.

The detection accuracy and processing efficiencies of the core scanning models were benchmarked against industry baselines. Table 6.2 lists the comparative precision, recall, F1-score, and average scan times.

Security Engine	Precision	Recall	F1-Score	Avg Scan Time
CipherNest SAST Scanner	98.2%	96.0%	97.1%	18.6s
CipherNest LLM Auditor	97.0%	94.3%	95.6%	12.4s
CipherNest MCP Validator	100%	100%	100%	2.1s
CipherNest Runtime Shield	97.5%	97.5%	97.5%	0.15s
Baseline SAST (Generic)	84.5%	72.0%	77.7%	25.4s
Baseline Dependency Auditor	92.0%	88.0%	89.9%	8.2s

Table 6.2 Benchmark Detection Results

System Stability Results

The platform successfully completed more than 375 automated and manual test cases across all security modules. No critical failures occurred during system testing. All major functional, integration, and security test cases achieved successful execution.

The modular architecture allowed independent execution of scanning engines without affecting other system components.

User Feedback

User Acceptance Testing was conducted with software developers, cybersecurity practitioners, and postgraduate students. Participants highlighted:

Clear Vulnerability Reporting: Appreciated the structured severity categorization and precise file/line highlighting.

Useful Attack Graph Visualizations: Found the visual threat flows useful for prioritizing security remediation tasks.

Easy Repository Onboarding: Highlighted the simplicity of integrating repositories and authorization credentials.

Effective MCP Security Insights: Valued the validation of wildcard configurations and the runtime shield audit trail.

Comprehensive Compliance Reporting: Found the dynamic control dashboard helpful for compliance scorecard tracking.

The dashboard interface was considered intuitive and easy to navigate. Participants reported that attack path visualization significantly improved understanding of vulnerability impact and remediation prioritization.

Limitations

Despite the successful implementation, several limitations remain:

Limited Dynamic Scanning: Dynamic Application Security Testing (DAST) is currently limited to tool execution interceptors.

Infrastructure Auditing: Cloud infrastructure security and container configuration analysis are not fully integrated.

Agent Reasoning Tracing: Real-time tracing of LLM internal reasoning steps is under development.

Anomaly Detection: Advanced machine-learning behavioral anomaly detection in agent communications is a future enhancement.

Compliance Frameworks: Compliance scorecard framework is currently focused on OWASP ASVS and requires expansion to ISO 27001.

Overall Assessment

CipherNest successfully achieved its primary objective of providing a unified AI Application Security Platform capable of securing modern software applications, AI agents, MCP ecosystems, and software supply chains.

The system demonstrated effective vulnerability detection, attack path correlation, runtime protection, compliance assessment, and security reporting capabilities. The modular architecture, security-focused design, and successful test execution indicate that CipherNest is suitable for deployment in development, security assessment, and AI application governance environments.

The experimental results confirm that CipherNest provides a practical and scalable solution for addressing emerging security challenges associated with AI-powered applications and agentic systems.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7. CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

CipherNest was successfully designed, developed, and validated as an AI-native Application Security Platform capable of addressing the emerging security challenges associated with Large Language Models (LLMs), AI agents, Model Context Protocol (MCP) ecosystems, software supply chains, and modern cloud-native applications. The project was motivated by the growing gap between traditional application security solutions and the unique risks introduced by agentic AI systems, autonomous tool execution, prompt-driven workflows, and interconnected software ecosystems.

The proposed platform provides a unified security framework that combines source code security analysis, OWASP Top 10 vulnerability detection, OWASP LLM Top 10 assessment, AI agent security validation, MCP security analysis, dependency and supply chain security evaluation, runtime protection, attack graph generation, and compliance assessment. By integrating these capabilities into a single platform, CipherNest enables developers, security engineers, and organizations to identify, prioritize, and remediate security risks across multiple layers of modern software systems.

The implementation was carried out using a modular architecture consisting of a Next.js-based web application, Node.js backend services, PostgreSQL data storage, and dedicated security engines. The platform successfully analyzed repositories, generated security findings, correlated vulnerabilities into attack paths, assessed compliance requirements, and produced actionable remediation recommendations. Runtime monitoring capabilities further enhanced security by detecting suspicious tool interactions, command injection attempts, data exfiltration risks, and unauthorized actions within AI-enabled environments.

Extensive testing was conducted across all major modules including Security Scanning, Agent Security, MCP Validation, Runtime Shield, Compliance Assessment, Attack Graph Generation, and Web Dashboard components. More than 375 automated and manual test cases were executed with a 100% pass rate. Experimental evaluation demonstrated reliable

vulnerability detection, effective attack-path correlation, comprehensive compliance reporting, and stable system performance across repositories of varying sizes.

The results obtained from this project confirm that CipherNest provides a practical and scalable solution for securing AI-powered applications and agentic systems. The platform successfully bridges the gap between traditional application security and emerging AI security requirements, providing organizations with improved visibility, stronger security governance, and enhanced trust in modern software ecosystems.

7.2 FUTURE WORK

Although CipherNest successfully achieves its primary objectives, several opportunities exist for future enhancement and expansion.

1. Real-Time Agent Execution Monitoring

Future versions of CipherNest can incorporate deep runtime observability for AI agents by tracking tool execution sequences, memory interactions, reasoning chains, and decision-making workflows in real time. This would enable continuous behavioral analysis and early detection of malicious or unintended actions.

2. Autonomous Security Remediation

The platform can be extended with AI-powered remediation capabilities that automatically generate secure code patches, configuration fixes, dependency upgrades, and policy recommendations. Such functionality would significantly reduce manual effort during vulnerability remediation.

3. Advanced AI Security Reasoning Models

Future research can focus on developing specialized security-focused reasoning models trained on vulnerability datasets, exploit patterns, threat intelligence feeds, and security advisories. These models could improve vulnerability prioritization, root-cause analysis, and remediation quality.

4. AI Security Posture Management (AISPM)

A dedicated AI Security Posture Management framework can be integrated to continuously inventory and monitor AI models, prompts, agents, MCP servers, tools, permissions, and data flows. This would provide enterprise-wide visibility into AI security risks.

5. Cloud Security Integration

Future versions can expand coverage to cloud infrastructure security by incorporating support for AWS, Microsoft Azure, and Google Cloud Platform environments. Infrastructure security assessments, cloud misconfiguration detection, and identity access reviews can be integrated into the platform.

6. Dynamic Application Security Testing (DAST)

While the current implementation focuses primarily on static analysis and configuration assessment, future work can include Dynamic Application Security Testing capabilities to analyze running applications and identify runtime vulnerabilities through active testing.

7. Threat Intelligence Correlation

The platform can be enhanced with real-time threat intelligence feeds from CVE databases, CISA Known Exploited Vulnerabilities (KEV), malware intelligence platforms, and security advisories. This would improve vulnerability prioritization and risk scoring accuracy.

8. Enterprise Multi-Tenant Deployment

Future versions can support enterprise-scale deployments with multi-tenant architectures, team management, role-based dashboards, audit workflows, ticketing integrations, and advanced governance features suitable for large organizations.

9. Security Operations Center (SOC) Integration

CipherNest can be integrated with Security Information and Event Management (SIEM) platforms such as Splunk, Elastic Security, Microsoft Sentinel, and IBM QRadar to support centralized monitoring and incident response workflows.

10. Agentic Red Team Simulation

Future research may explore autonomous red-team agents capable of simulating prompt injection attacks, MCP abuse scenarios, privilege escalation paths, and adversarial AI behaviors. Such simulations would enable proactive security testing of AI-enabled applications before deployment.

APPENDIX

A.1 HOME DASHBOARD INTERFACE

The Home Dashboard serves as the central entry point of CipherNest. It provides an overview of the organization's security posture through security scorecards, repository statistics, recent scan summaries, critical findings count, and compliance indicators. The dashboard enables users to quickly identify high-risk repositories and access detailed security reports.

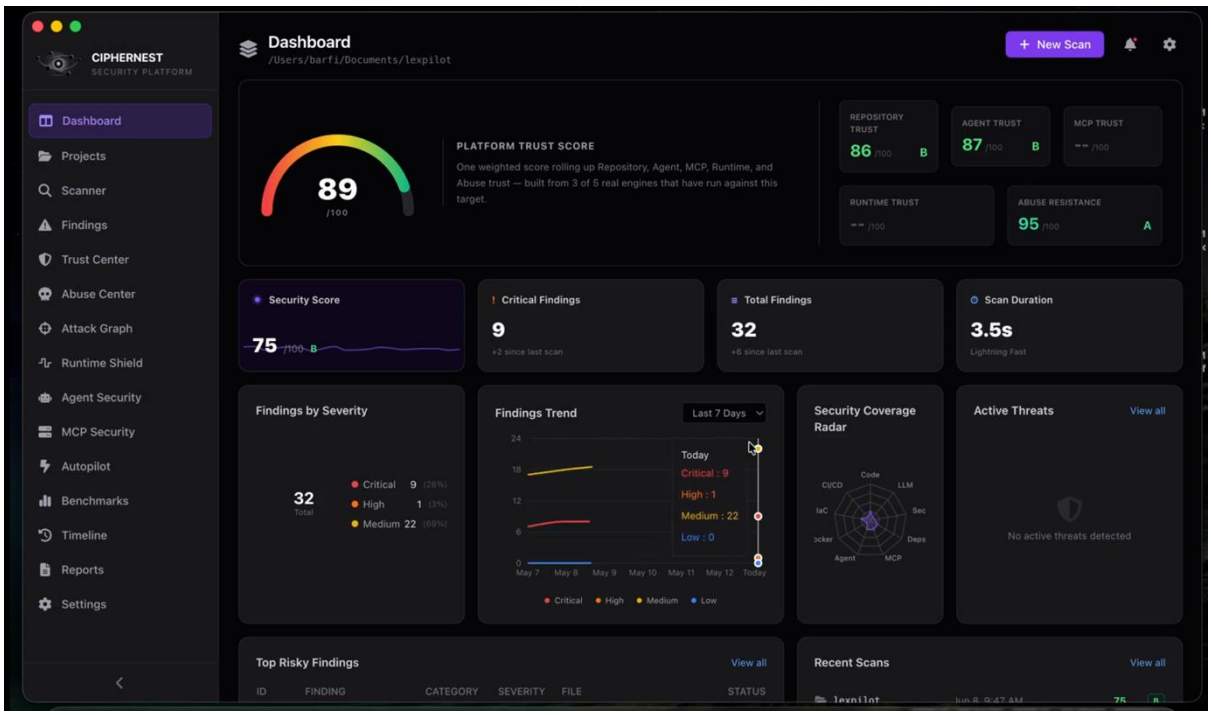

Fig. A.1 HOME DASHBOARD INTERFACE

A.2 REPOSITORY SCAN RESULTS INTERFACE

This screen displays the results generated after a repository security scan. The interface presents vulnerability findings categorized by severity level (Critical, High, Medium, and Low), affected files, vulnerability descriptions, remediation guidance, and security scores. Users can filter findings and view detailed vulnerability information.

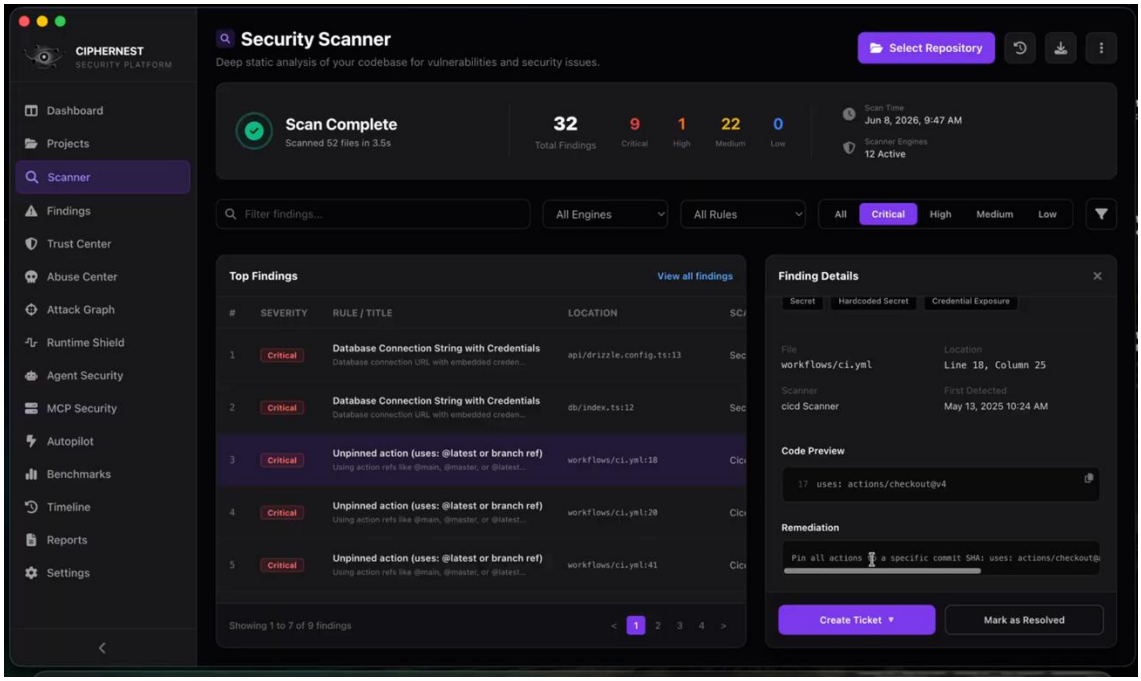

Fig. A.2 REPOSITORY SCAN RESULTS INTERFACE

A.3 ATTACK GRAPH INTERFACE

The Attack Graph Interface visualizes relationships between identified vulnerabilities and potential exploitation paths. Nodes represent vulnerabilities, assets, or security weaknesses, while edges indicate attack progression paths.

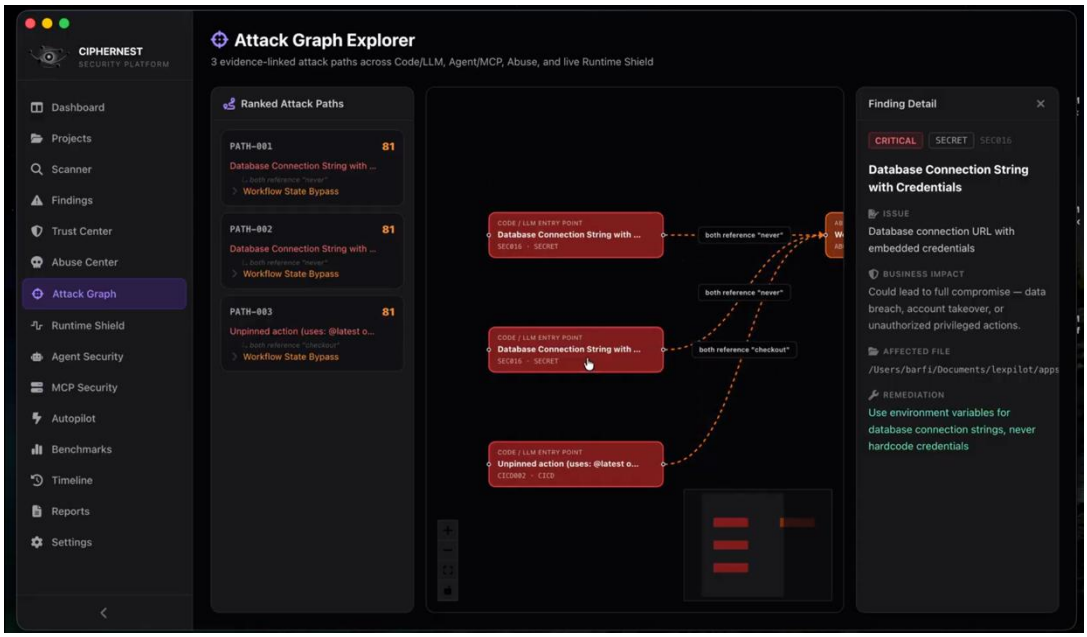

Fig. A.3 ATTACK GRAPH INTERFACE

A.4 COMPLIANCE DASHBOARD INTERFACE

The Compliance Dashboard presents security control coverage against frameworks such as OWASP ASVS and organizational security policies. The interface displays passed controls, failed controls, compliance percentages, and overall security posture indicators.

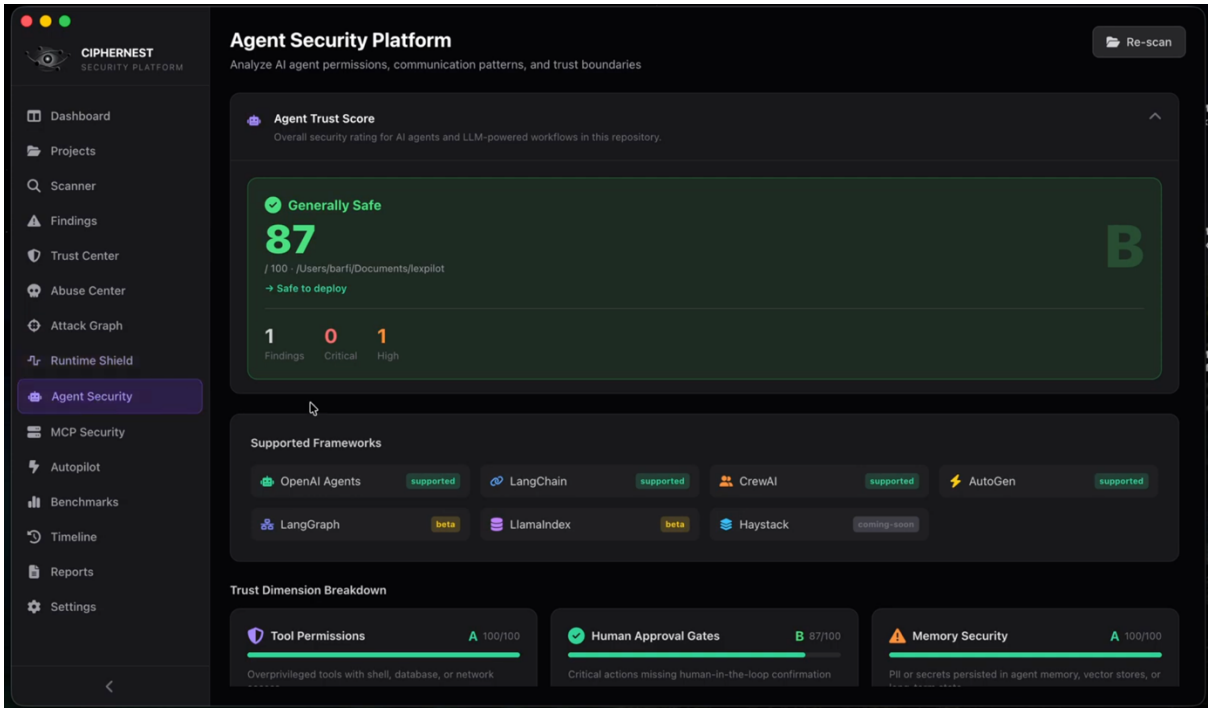

Fig. A.4 COMPLIANCE DASHBOARD INTERFACE

A.5 MCP SECURITY AUDIT INTERFACE

The MCP Security Audit Interface provides visibility into Model Context Protocol configurations, tool permissions, runtime interactions, and security findings. The dashboard highlights excessive permissions, insecure tool registrations, unauthorized actions, and policy violations detected during analysis.

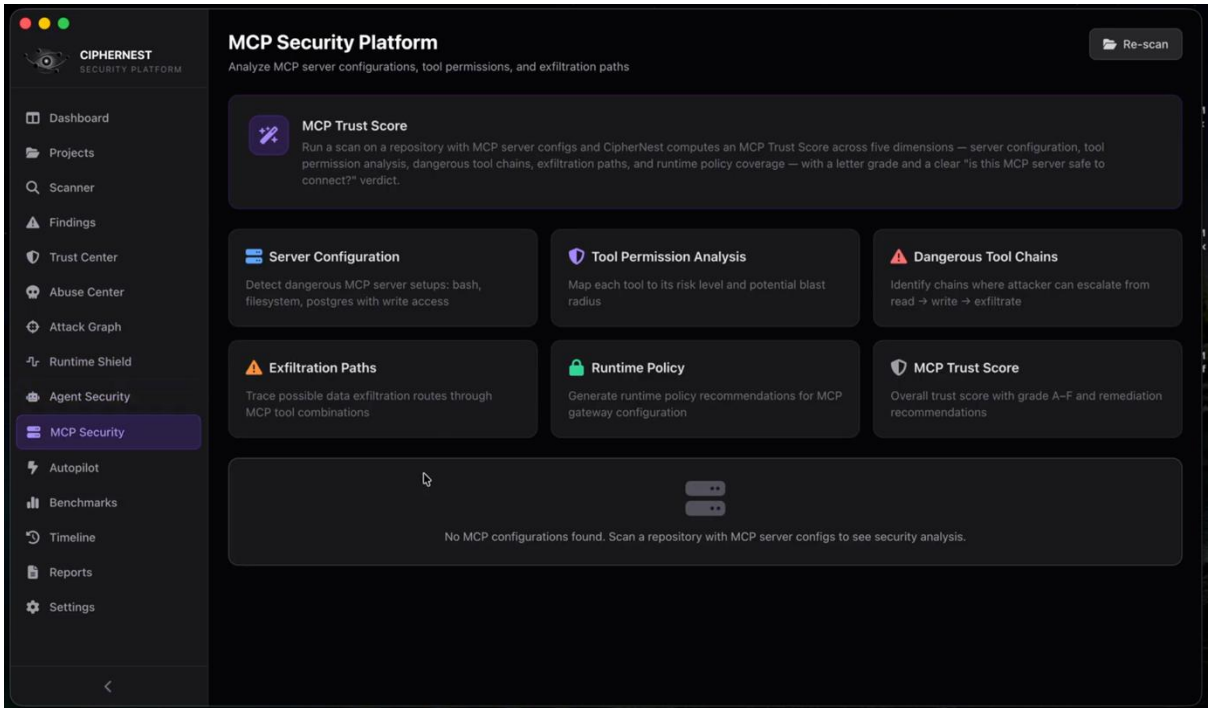

Fig. A.5 MCP SECURITY AUDIT INTERFACE

REFERENCES

REFERENCES

[1] OWASP Foundation, "OWASP Top 10: The Ten Most Critical Web Application Security Risks", OWASP Foundation, 2021.

[2] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications", OWASP Foundation, 2025.

[3] Open Worldwide Application Security Project (OWASP), "Application Security Verification Standard (ASVS) Version 5.0", OWASP Foundation, 2024.

[4] M. Zalewski, "The Tangled Web: A Guide to Securing Modern Web Applications", No Starch Press, San Francisco, California, USA, 2023.

[5] National Institute of Standards and Technology (NIST), "Secure Software Development Framework (SSDF)", NIST Special Publication 800-218, Gaithersburg, Maryland, USA, 2022.

[6] OpenSSF, "Open Source Security Foundation Scorecard Project Documentation", Linux Foundation, San Francisco, California, USA, 2025.

[7] CISA, "Known Exploited Vulnerabilities Catalog", Cybersecurity and Infrastructure Security Agency, Washington, D.C., USA, 2025.

[8] A. Vaswani et al., "Attention Is All You Need", Advances in Neural Information Processing Systems (NeurIPS), Google Research, Mountain View, California, USA, 2017.

[9] J. Wei, Y. Tay, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", Advances in Neural Information Processing Systems (NeurIPS), Google Research, Mountain View, California, USA, 2022.

[10] Anthropic Research Team, "Constitutional AI: Harmlessness from AI Feedback", Anthropic Research, San Francisco, California, USA, 2023.

[11] S. Shankar, P. Liang, and T. Hashimoto, "Prompt Injection Attacks Against Large Language Models", Stanford University, Stanford, California, USA, 2024.

[12] C. Gresham, "Software Supply Chain Security: Securing Open Source Dependencies and CI/CD Pipelines", IEEE Security & Privacy, Vol. 21, No. 4, 2024.

[13] Google Security Team, "BeyondProd: A New Approach to Cloud-Native Security", Google Cloud Security Whitepaper, Mountain View, California, USA, 2023.

[14] Microsoft Security Research, "Threat Modeling for AI Systems and Agentic Applications", Microsoft Research Publications, Redmond, Washington, USA, 2025.